



UNIVERSIDADE D
COIMBRA

Francisco Alexandre Antunes

**HUMAN GAIT ANALYSIS USING AN INSTRUMENTED
KNEE BRACE**

**Master's Dissertation in Electrical and Computer Engineering, Computer
field, supervised by Professor António Paulo Mendes Breda Dias Coimbra
and Professor João Paulo Morais Ferreira, presented to the Faculty of Science
and Technology of the University of Coimbra.**

July 2024



UNIVERSIDADE D
COIMBRA

**Human Gait Analysis using an Instrumented
Knee Brace**

Francisco Alexandre Antunes

A dissertation presented for the degree of
Master of Electrical and Computer Engineering in the
Computer field supervised by Prof. António Paulo
Mendes Breda Dias Coimbra and Prof. João Paulo
Morais Ferreira

Department of Electrical and Computer Engineering
Faculty of Science and Technology of the University of
Coimbra
Portugal
January 2025

*“We are continually faced by great opportunities brilliantly disguised as
insoluble problems.”
—Lee Iacocca*

Acknowledgements

Em primeiro lugar, quero expressar os meus agradecimentos aos meus orientadores Professor Doutor António Paulo Mendes Breda Dias Coimbra e Professor Doutor João Paulo Morais Ferreira por me guiarem e ajudarem ao longo da realização deste projeto. Contribuíram não só para o meu desenvolvimento a nível académico e profissional mas também para o término desta etapa bastante importante. Agradecer também ao engenheiro João Lourenço do ISR pelo auxílio prestado no desenvolvimento das peças 3D usadas.

Quero agradecer aos meus amigos de Coimbra por me ajudarem ao longo destes anos enquanto estudante, por me motivarem a chegar mais longe e que mesmo seguindo caminhos diferentes neste percurso académico, sempre estiveram lá para mim, para sempre serão uma segunda família.

Aos meus amigos que vêm desde os tempos da primária e aos que fiz ao longo do meu percurso escolar antes de entrar na etapa universitária, que mesmo não estando presentes no meu dia a dia, não deixaram de ser suporte e motivo de alegria quando reunidos.

Por último, mas não menos importante, quero agradecer à minha família por me ensinarem o que é preciso para ser a pessoa que sou hoje. Por todos os momentos que me proporcionaram até agora, que me tornaram na pessoa que sou hoje. Mas em especial aos meus pais e ao meu irmão. Um agradecimento especial ao pai pelos sacrifícios que tomou para me proporcionar a vida que tive até agora, à minha mãe por estar sempre ao meu lado tanto nos momentos bons como nos maus, e por nunca me ter deixado desistir dos desafios que fui a encontrar ao longo da vida e ao meu irmão em quem tenho muito orgulho e sempre me motivou para ser melhor tanto a nível académico como pessoal. Também um agradecimento especial à minha madrinha que sempre será uma segunda mãe para mim, que cuidou de mim e esteve sempre ao meu lado desde o início.

A todos, muito obrigado.

Abstract

Human gait can often be affected by pathologies caused by neuromuscular and/or musculoskeletal injuries. The analysis and diagnosis of these pathologies is an important process in the rehabilitation of each patient. To this end, various systems have been developed and optimised over the years.

For this dissertation, a system consisting of an Instrumented Knee Brace (IKB) and a diagnostic application that collects and displays data on the variance of the knee flexion angle during gait was developed. The data collected by the encoder is transmitted via Message Queuing Telemetry Transport (MQTT) to the application, allowing the system to work without the need for cable connections between the instrumented knee brace and the user's computer. The IKB is equipped with an absolute rotary encoder to read the knee angle variation during gait.

The application was developed with the aim of being an intuitive tool to help analyse and diagnose human gait, allowing interactive analysis of the data obtained, presenting it graphically and allowing the user to observe small time intervals alternately. It also shows a curve that is considered normal, generated from a deep learning model, using personal data about the patient such as age, weight, height, and gender. This allows a comparison to be made between the values read by the encoder and the values generated by the model. The values obtained by the encoder are stored in a database, allowing these values to be analysed in the future after the session. The database contains information about each user, patient, and session carried out.

Keywords: Encoder, Wireless, Instrumented Knee Brace, Human Gait, Deep Learning

Resumo

A marcha humana é frequentemente afetada por patologias causadas, lesões neuromusculares e/ou músculo-esqueléticas. A análise e diagnóstico destas patologias é um processo importante na reabilitação de cada paciente, para tal vários sistemas foram desenvolvidos e otimizados ao longo dos anos.

Nesta dissertação foi desenvolvido um sistema constituído por uma joelheira instrumentada e uma aplicação de diagnóstico que recolhe e apresenta dados referentes à variância do ângulo de flexão do joelho ao longo da marcha. Os dados recolhidos pelo codificador são transmitidos via MQTT para aplicação, permitindo que o sistema funcione sem ter de haver ligações por cabo entre a joelheira instrumentada e o computador do utilizador. A joelheira instrumentada é equipada por um codificador rotativo absoluto para ler a variação do ângulo do joelho durante a marcha.

A aplicação foi desenvolvida com o objetivo de ser uma ferramenta de uso intuitivo para auxílio na análise e diagnóstico da marcha humana, permitindo uma análise interativa com os dados obtidos, apresentando-os graficamente e permitindo que o utilizador observe pequenos intervalos de tempo alternadamente. Ainda é apresentada uma curva considerada normal gerada a partir de um modelo de aprendizagem computacional, usando dados pessoais sobre o paciente como a idade, peso, altura e género. Isto permite fazer uma comparação entre os valores lidos pelo codificador e os valores gerados pelo modelo. Os valores obtidos pelo codificador são guardados numa base de dados, permitindo a análise desses valores num futuro após a sessão. A base de dados contém informação sobre cada utilizador, paciente e sessão realizada.

Palavras Chave: Codificador, Comunicação Sem Fios, Joelheira instrumentada, Marcha Humana, Aprendizagem Profunda

Contents

List of Figures	V
List of Tables	VI
List of Acronyms	VII
1 Introduction	1
1.1 Contributions	1
1.2 Structure	2
2 Literature Review	4
2.1 Human Gait	4
2.2 Knee joint	6
2.3 Gait Analysis	8
2.4 Related Works	9
2.4.1 Motion Capture Systems	9
2.4.2 Computational modelling	10
2.4.3 Clinical Applications	11
2.4.4 Recent Developments	11
2.5 Final Considerations	13
3 Protocols and frameworks	14
3.1 MQTT	15
3.2 Python and Django for Web Development	18
4 Hardware and Software Development	20
4.1 Subject-specific Knee Flexion Curve Generation	20
4.1.1 Data Selection	20
4.1.2 Model train and test	21
4.2 Instrumented Knee Brace	24
4.2.1 Arduino MKR Wi-Fi 1010	24
4.2.2 Encoder	26
4.2.3 Knee Brace	29

4.2.4	System Development	29
4.3	Data Acquisition and Transmission System	33
4.4	Web Application	34
5	Test and Results	38
5.1	Test procedure	38
5.2	Calibration	40
5.3	Tests	41
6	Conclusion and Future Work	45
6.1	Conclusion	45
6.2	Future Work	46
A	Web Application layout	47
B	Arduino code	51

List of Figures

2.1	Gait phases	5
2.2	Gait sub-phases	6
2.3	Knee Structure	7
2.4	Gait data from a 15-year-old male with a diagnosis of cerebral palsy	8
2.5	Normal knee flexion angle during one gait cycle	9
2.6	Motion Capture System	10
2.7	Leg and ankle musculoskeletal model	10
2.8	Schematic of implementation of Inertial Measurement Units (IMUs) (yellow rectangles) for gait analysis	12
2.9	IMUs implementation in	12
2.10	Digital Goniometer with magnetic encoder	13
3.1	System architecture	14
3.2	publish-subscribe process in MQTT	15
4.1	Model structure generated by using <i>tensorboard</i> python library	22
4.2	Training and Validation loss	22
4.3	Comparison between generated and real curves from the test set	23
4.4	Dynamic Time Warping (DTW) vs Euclidean distance	24
4.5	Arduino MKR Wi-Fi 1010	25
4.6	CUI AMT222B-V	29
4.7	Knee Brace with Lateral Reinforcements	30
4.8	Knee Brace Lateral Reinforcements	30
4.9	3D Printed Components: (1) Grommet, (2) Shaft, (3) Encoder Support, (4) Movement Blocker	31
4.10	System Architecture: (a) encoder, (b) RJ12 Connector, (c) RJ12 Cable, (d) Arduino MKR 1010, (e) Buzzer, (f) Green LED, (g) 330Ω Resistor	32
4.11	Final system	32
4.12	User main page	35
4.13	Patient information page	36
4.14	Session review page	36

4.15	New session page	37
5.1	Calibration procedure	40
5.2	Session results and curve comparisons for first subject	42
5.3	Session results and curve comparisons for second subject	43
5.4	Session results and curve comparisons for third subject	44
A.1	Login Page	47
A.2	Main User Page	48
A.3	Patient Information Page	48
A.4	New Session Page	49
A.5	Session Review Page	50

List of Tables

4.1	Test metrics scores	24
4.2	Pin Connections between Encoder, Arduino MKR 1010 and RJ12 Connector	31
5.1	Obtained angle values	41

List of Acronyms

EMG	electromyography
IMUs	Inertial Measurement Units
kg	kilogram
cm	centimeters
IKB	Instrumented Knee Brace
ACL	Anterior Cruciate Ligament
PCL	Posterior Cruciate Ligament
QoS	Quality of Service
MQTT	Message Queuing Telemetry Transport
PubAck	publish acknowledgment
PubRec	publish received
PubRel	publish release
PubComp	publish complete
JS	JavaScript
SPI	Serial Peripheral Interface
MLP	Multi Layer Perceptron
DTW	Dynamic Time Warping

Chapter 1

Introduction

This dissertation presents a web application to help diagnose potential knee injuries by analysing human gait by using a developed IKB. The aim is to determine the variation of the angles of the knees during a gait cycle and compare it to a subject-specific knee flexion curve generated by a deep learning model.

The term “gait” is defined as the rhythmic motion of the lower limbs that results in the forward progression of the body with minimal energy expenditure. Gait analysis is a technique used by clinicians and researchers to assess and treat individuals with conditions, pathologies, or injuries that affect their ability to walk or run. A comprehensive understanding of the gait cycle and its phases is essential for achieving positive patient or athlete outcomes[1].

To address the limitations of conventional diagnostic methods, an IKB was developed. It uses data from the knee angle collected during a person’s gait cycle to compare with a subject-specific gait cycle and provide a preliminary diagnosis.

1.1 Contributions

The dissertation work first began with the generation of a subject-specific normal knee angle curve based on objective personal patient characteristics, including sex, age, height (in centimetres), and weight (in kilogram). To achieve this objective, a neural network was developed and trained using data from a variety of datasets that contain knee angle variation curves and information about each subject that participated in each data collection session. The subsequent step was the development of the IKB hardware. An encoder was employed to measure the knee angles during a gait session.

Following the training of the model, and the development of the IKB, a web application was developed to present the knee curves obtained by the encoders and the one generated by the deep learning model to the user.

After everything worked properly, tests were done to validate the system developed.

1.2 Structure

In this section it is presented the structure of the thesis.

- **Chapter 2: Literature Review:**

- Comprehensive review of existing knowledge related to human gait and knee joint mechanics.
- Covers phases of the gait cycle, anatomy, and function of the knee joint, and common knee injuries like ACL and meniscal tears.
- Discusses the importance of gait analysis in diagnosing and treating gait disorders.
- Reviews various methods and devices used in clinical gait analysis, including force platforms, computational modelling, and recent technological advancements.

- **Chapter 3: Protocols and framework**

- Explanation of what is MQTT
- Explanation of what is Django

- **Chapter 4: Hardware and Software Development:**

- Details the technical aspects of the work developed.
- Process of developing a neural network that generates subject-specific knee flexion curves using data from multiple datasets.
- Covers the development of the IKB, including the selection of components such as the Arduino MKR Wi-Fi 1010 and an absolute rotary encoder.
- Explains the data acquisition and transmission system, focusing on the use of **mqtt!** (**mqtt!**) for wireless communication.
- Describes the development of the web application for visualizing and

interacting with the data.

- **Chapter 5: Test and Results:**

- Presents the methodologies used to test and validate the hardware and software components.
- Describes testing procedures and criteria for evaluating performance of the IKB.
- Discusses findings, highlighting accuracy and reliability of the system in measuring knee angles and analysing gait.

- **Chapter 6: Conclusion and Future Work:**

- Recapitulates research findings, emphasizing contributions of the IKB to gait analysis.
- Summarizes effectiveness of the developed system in diagnosing knee injuries through gait analysis.
- Provides recommendations for future research, suggesting potential improvements and further applications of the developed technology.

Chapter 2

Literature Review

This chapter presents a review of pertinent literature on human gait analysis, with a particular focus on the influence of the knee joint on potential gait abnormalities. To provide a brief overview of the dissertation's objective, a summary of the concept of human gait and the gait phases is presented.

2.1 Human Gait

Human gait is a complex cycle of alternating movements of the lower extremities. The cycle begins with heel strike, when one foot strikes the ground after a swing, and ends with the next heel strike of the same foot [2]. Gait can be measured using kinetic (force) or kinematic (spatial/temporal) information. A standard gait assessment requires at least one gait cycle [1].

Gait Phases

A gait cycle consists of two global phases: the stance phase and the swing phase.

1. The **stance phase** starts when the foot is in contact with the ground and ends when it leaves the ground.
2. The **swing phase** is the opposite of the stance phase. It begins when the foot leaves the ground and ends when the same foot makes contact with it.

The stance phase makes up 60% of the gait cycle, with the swing phase making up the remaining 40%.

In addition to the two previously mentioned gait phases, there exist three

support phases presented in figure 2.1 [3]:

1. **Single support phase** also known as the swing phase where only one limb is in contact with the floor. In normal gait, it comprises between 60% and 72% of the stance phase.
2. **Initial double support phase** is the sub-phase between heel contact and contralateral foot-off, comprising approximately 14% to 20% of the stance phase.
3. **Terminal double support phase** is the sub-phase from contralateral foot-on to toe-off, comprising approximately 14% to 20% of the stance phase. The total double support phase is defined as the sum of the initial and terminal double support phases, which make up approximately 28% to 40% of the stance phase.

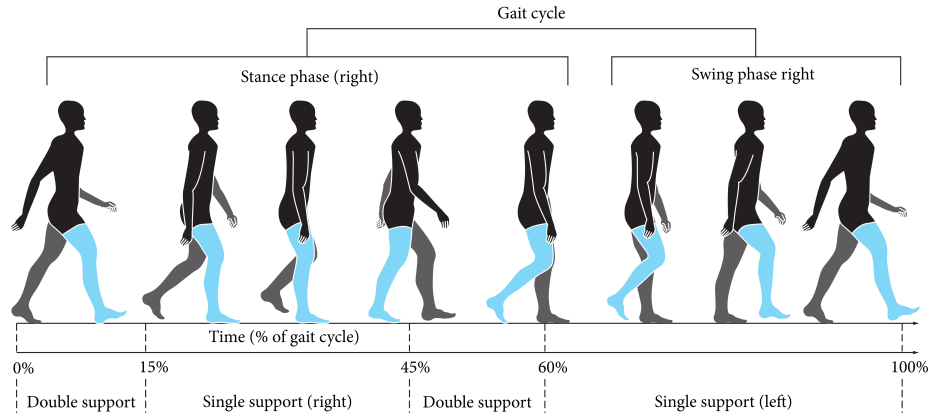


Figure 2.1: Gait phases
[3]

Gait pathologies typically appear during the stance phase where the foot is loaded, which impacts the swing phase. In order to better understand the foot function, the stance phase can be divided into four sub-phases called [1]:

1. **Contact phase** (heel contact or heelstrike), makes up for 14% to 20% of the stance phase. It is represented by the first heel contact of the lead leg and the 'toe-off' of the contralateral leg
2. **Foot-flat phase**, makes up for 16% to 22% of the stance phase. It has an important function since the body weight is transferred to the lead leg in this sub-phase, and must absorb this weight while maintaining momentum.
3. **Mid-stance phase**, makes up for 29% to 37% of the stance phase. In this sub-phase the support limb moves from shock absorption to a stability

function. It is represented from the '*toe-off*' point of the contralateral leg to the first point the heel stops making contact with the floor.

4. **Propulsive or Pushoff phase** (passive or active), makes up for 45% to 55% of the stance phase. The propulsive phase is divided into active and passive components, which respectively account for 31% to 35% and 14% to 20% of the phase. During the propulsive phase, the foot usually supinates to enable a more rigid mid-foot, allowing it to act as a lever (Winlass mechanism) to propel the body forward. The propulsive phase of gait can be further divided into an active propulsive and passive propulsive phase to isolate the component of the propulsive phase while still in single support, and the passive component of the propulsive phase when in double support.

This sub-phases are represented in figure 2.2 during a complete gait cycle.

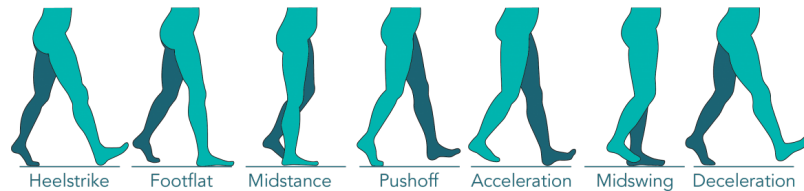


Figure 2.2: Gait sub-phases
[1]

2.2 Knee joint

The knee joint is a complex synovial joint comprising three articulations: one between the patella and femur, and the other two between the tibial and femoral condyles.

The knee joint is capable of two main movements: flexion and extension on a horizontal axis, and some degree of rotation called locking and unlocking on a vertical axis. The evolution of the human body has resulted in an erect posture, which provides the advantage of standing on two legs. This requires the weight of the upper body to be balanced and carried by the lower limbs, placing significant stress on the knee joint. The constant stress and wear and tear caused by this activity disrupts the internal balance of the knee joint [4].

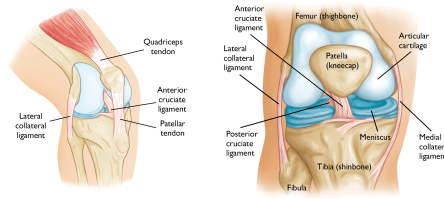


Figure 2.3: Knee Structure
[5]

The most common knee injuries like Anterior Cruciate Ligament (ACL) injury, Posterior Cruciate Ligament (PCL) injury and meniscal tears, include tear of soft tissues, fractures, and dislocations, and in many cases involve more than one structure in the knee.

Some common injuries are [5]:

- **ACL injuries:** ACL injuries are a significant problem, especially for athletes involved in high-intensity sports. The ACL is one of the key ligaments that help stabilize the knee joint and is often injured during activities that involve sudden changes in direction, cutting, pivoting, or improper landing from a jump. Sports such as soccer, football, and basketball are particularly associated with a higher risk of ACL injury due to the dynamic and high-impact movements involved.
- **PCL injuries:** PCL injuries are less common than ACL injuries, but can still significantly impact knee stability and function. The PCL is one of the key ligaments in the knee that helps control the posterior motion of the tibia (shin-bone). PCL injuries often result from a direct blow to the front of the knee while the knee is bent, a common scenario in motor vehicle accidents and sports-related contact.

Unlike ACL injuries, PCL tears tend to be partial and have a greater potential to heal on their own. Symptoms of a PCL injury may include pain, swelling, and a feeling of instability in the knee, although these symptoms may be less severe than those associated with ACL injuries.

- **Meniscal Tears:** Meniscal tears are a common injury, especially among athletes. These tears often occur during activities that involve twisting, cutting, pivoting, or being tackled, such as sports like football, soccer, or basketball. However, meniscal tears are not limited to athletes. They can also result from degenerative changes due to arthritis or the natural ageing process. As people age, the menisci can weaken and become more susceptible to injury. In these cases, even a small movement, such as an awkward twist when getting up from a chair, can be enough to cause a

tear. This type of tear is often called a degenerative meniscus tear.

2.3 Gait Analysis

Clinical gait analysis provides objective and reliable biomechanical information, including temporal wave forms for each of the lower body joints. There are a lot of different types of measurement devices like 3D motion capture, force plates, instrumental mats, wearable sensors with inertial measurements and accelerometer [6].

Gait disorders are unusual walking or running patterns that can be caused by injury or disease. Usually, these pathologies cause pain in joints such as knees, hips, ankles, or back pain. The main aim of gait analysis is to evaluate the way an individual stands and walks and help to

- Identify the source of muscle, nerve or skeletal problems
- Discover the source of a patient's pain while walking or simply standing
- Help to diagnose bone deformities or skeletal misalignments
- Assist in discovering muscle or nerve dysfunctions
- Check the progress of walking related diseases

Gait analysis is an excellent way to identify certain medical conditions, determine whether further tests are needed, and suggest possible treatment options [7].

Knee Analysis During Gait

Gait pathologies like knee extensor muscle weakness can make it harder for an individual to support himself during the stance phase [8].

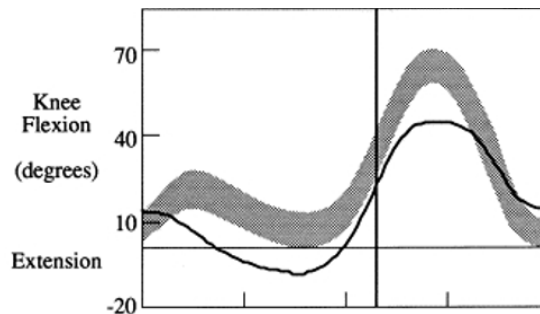


Figure 2.4: Gait data from a 15-year-old male with a diagnosis of cerebral palsy [9]

Figure 2.4 illustrates the discrepancy between the typical range of knee flexion during gait and the observed range in a 15-year-old male with cerebral palsy. The peak knee flexion is notably diminished in comparison to the typical gait pattern. This analysis enables the identification of gait abnormalities. By integrating additional data, such as electromyography (EMG) readings from other joints, it is possible to assess the extent of injury and develop an effective treatment plan [9].

In figure 2.5 is represented normal values for knee flexion during a gait cycle retrieved from the *Multivariate Gait Data* dataset [10]. These values may vary between individuals depending on characteristics such as weight, height, age, gender, gait velocity, stride length and other factors.

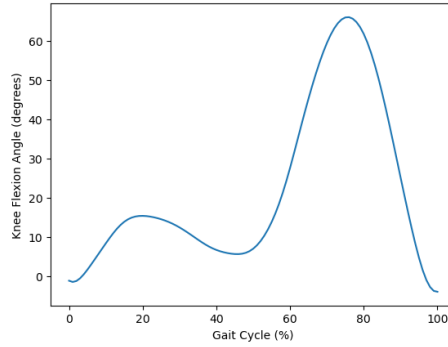


Figure 2.5: Normal knee flexion angle during one gait cycle [10]

2.4 Related Works

2.4.1 Motion Capture Systems

The advent of motion capture technology has revolutionised gait analysis. Vicon systems, which were first introduced in the 1980s, enabled the precise three-dimensional tracking of reflective markers placed on the body [11]. This technology allowed for the detailed quantification of joint angles, velocities, and accelerations, facilitating the development of more sophisticated biomechanical models. Integration of these platforms with motion capture systems provided a comprehensive representation of the external forces and moments acting on the body, which is crucial for elucidating the dynamics of gait.

The development of force platforms provided critical data on ground reaction forces during gait [12]. These platforms, when integrated with motion capture systems, provided a complete picture of the external forces and moments acting on the body, essential for understanding the dynamics of gait.

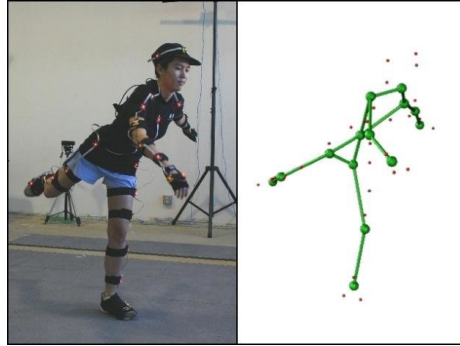


Figure 2.6: Motion Capture System presented in [13]

In figure 2.6 we can observe a motion capture system where on the left side of an actor using the system and on the right side is the 3D human pose inferred from the relative positions of each marker [13].

2.4.2 Computational modelling

The increased computational power available in the late 20th and early 21st centuries enabled researchers to develop complex musculoskeletal models for the simulation and analysis of gait. Scot L. Delp et al. created one of the first comprehensive musculoskeletal models, which has been continuously refined and remains widely used in research and clinical applications [14]. These models allow for the simulation of muscle forces, joint loads, and even the effects of surgical interventions as it is presented in figure 2.7.



Figure 2.7: Leg and ankle musculoskeletal model [14]

2.4.3 Clinical Applications

The application of gait analysis in clinical settings has profound implications. Jacquelin Perry outlined its use in diagnosing and treating gait abnormalities in patients with neuromuscular disorders[15]. Gait analysis is employed in the planning of surgical interventions, the design of rehabilitation programmes and the monitoring of the progress of treatments.

2.4.4 Recent Developments

- **Machine Learning and Gait Recognition**

The integration of machine learning techniques with gait analysis has recently been the subject of considerable research. These methods enhance the capability to identify and classify gait patterns for applications such as biometric identification and fall risk assessment [16]. Deep learning algorithms, in particular, have shown promise in automating the detection of gait abnormalities and improving the accuracy of predictive models [17]. Models based on auto-encoders can be advantageous for detection of pathologies like osteoarthritis in an early phase by analysing X-ray images [18].

- **Wearable Technology**

The increasing prevalence of wearable sensors has made gait analysis more accessible.

IMUs are used for gait analysis given that, with proper calibration, they are capable of measuring joint angles, linear velocity and angular velocity. This is achieved by using algorithms, incorporated by some commercially available devices, that provide estimations of the sensor's orientation. Those orientations can be represented by quaternions, rotary matrices and Euler angles [19].

In figure 2.9 it is possible to observe the methodology used in [20] to estimate the lower limb joint angles by using three encoders. One in the thigh, one in the shank and one in the foot. By performing some calculations, it is possible to obtain the angle formed by the different lower limb junctions during the human gait.

The use of encoders is also a subject of study in the field of human gait analysis. The use of this technology has the purpose, similar to the use of IMUs, to make the procedure of human gait analysis more affordable and reduce the dependency of special laboratories. In article [21] it was used a magnetic encoder, figure 2.10, to measure knee angles during gait.

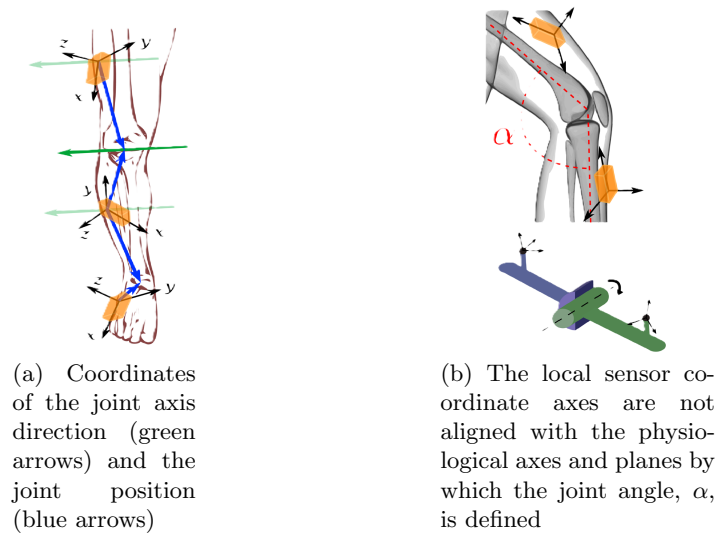


Figure 2.8: Schematic of implementation of IMUs (yellow rectangles) for gait analysis

[19]

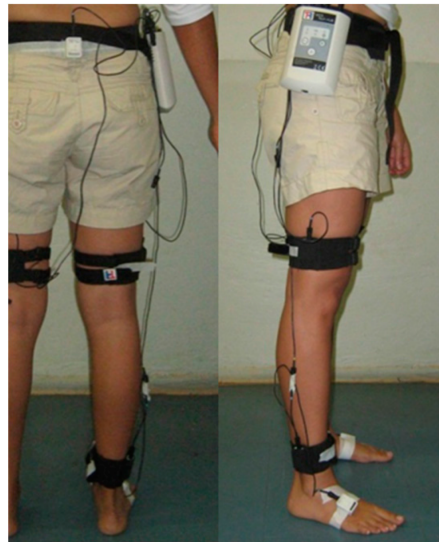


Figure 2.9: IMUs implementation in [20]

This shift towards portable, unobtrusive devices expands the potential for long-term monitoring and personalised healthcare.

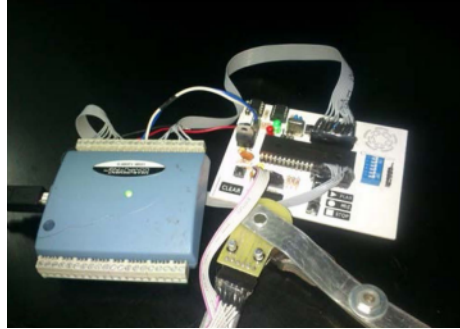


Figure 2.10: Digital Goniometer with magnetic encoder
[21]

2.5 Final Considerations

The analysis of human gait is a method of identifying any deviations between the gait performed and the considered “normal” gait for each individual. Given that there are no two individuals with the same characteristics and physical features, the analysis of each case is complex and requires the correct procedures to be followed.

Some systems, such as EMG and the analysis of joint variations during gait by using instruments such as IMUs and other motion capture systems, can assist with the analysis and diagnosis of potential injuries. The integration of machine learning into the domain of human gait analysis has recently enabled the more effective analysis of gait patterns. This is due to the fact that machine learning is capable of identifying and classifying gait patterns, as well as distinguishing them from patterns that can be considered “normal”.

The advancement of these systems can contribute to a more comprehensive understanding of the impact of injuries and pathologies on the human gait, thereby facilitating the process of their identification and the rehabilitation process for each individual.

The analysis of previous related works and articles provided insights that were invaluable in guiding the approach to be taken on the system developed in this dissertation.

Chapter 3

Protocols and frameworks

The dissertation comprises two systems that are connected by a message exchange protocol, as illustrated in figure 3.1. One of the primary objectives of this research was to develop a system that consumes minimal energy. Accordingly, a lightweight message exchange protocol that was reliable was selected.

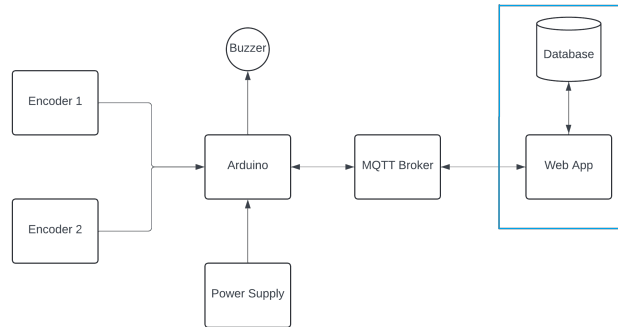


Figure 3.1: System architecture

In order to develop the web application, represented with the blue rectangle in figure 3.1, it was necessary for the programmer to have access to a framework that allowed its development. Given the necessity for a database in order to store the patient data to be collected, the Django framework was selected as it have incorporated an integrated database.

3.1 MQTT

MQTT is a lightweight publish-subscriber messaging protocol designed for efficient and reliable communications for resource-constrained devices and low bandwidth, high latency, or unreliable network. It is used mostly in IoT applications, since it provides efficient communication between peripherals and other devices [22].

This protocol is based on a client-server service. A client connects to a broker via a TCP/IP socket and from there can send or receive messages to or from the broker. The MQTT broker handles all interactions with the client connections and disconnections, subscription and unsubscription request and routing messages. Depending on the broker structure, it can handle multiple connections and large messages throughput.

The publish-subscribe pattern separates the client that sends messages from the client that receives messages, which is what distinguishes it from the client-server pattern [22].

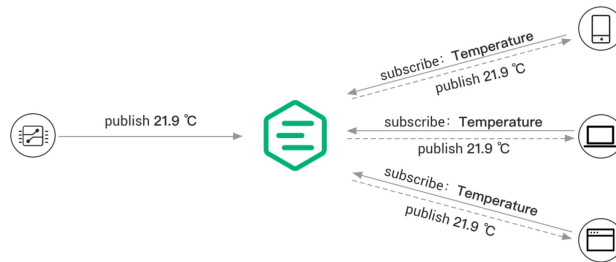


Figure 3.2: publish-subscribe process in MQTT [22]

Central to MQTT's functionality is its Quality of Service (QoS) mechanism, which ensures varying levels of reliability for message delivery.

The QoS levels in MQTT define the guarantee of message delivery between clients and the broker. Each level represents a different trade-off between reliability and resource consumption [23].

QoS 0: At Most Once

QoS 0, also known as *fire and forget*, ensures that a message is delivered at most once, with no acknowledgment from the receiver. This level of service does not guarantee message delivery, making it suitable for non-critical data where occasional loss is tolerable. The absence of acknowledgments reduces overhead, thereby minimizing latency and conserving bandwidth. Use cases for

QoS 0 include frequent sensor updates where missing a few readings does not significantly impact the application.

Mechanism

The client sends a message to the broker with QoS 0. It sends the message without storing it in any way or requiring the broker to acknowledge it. Neither the client nor the broker does anything to resend the message if it is lost in transit.

Use Cases

Typical use cases for QoS 0 include:

- Sensor data updates, where the occasional loss of data is acceptable.
- Telemetry data, where the latest reading is more important than the delivery of every single message.
- Non-critical notifications or status updates.

QoS 1: At Least Once

QoS 1 does not guarantee that messages will be delivered only once to the receiver. The sender delivers the message and waits for response from the broker. In case no acknowledgement is received within a specified period, the message will need to be sent again. This approach assures delivery but can lead to duplicate messages due to delayed or lost acknowledgements. For example, QoS 1 is well suited for applications where message delivery is important, but duplicates can be tolerated, such as status updates or control commands in a house automation system.

Mechanism

In QoS 1, the client sends the message and stores it until it receives a publish acknowledgment (PubAck) packet from the broker. If the PubAck is not received within a certain timeframe, the client resends the message. This process continues until the client receives the acknowledgment.

Use Cases

Typical use cases for QoS 1 include:

- Status updates that are critical but can tolerate duplicates.
- Control messages in home automation systems where commands must be received but duplicates can be ignored.

- Applications where network reliability varies and occasional message duplication is acceptable.

QoS 2: Exactly Once

The highest level of service, QoS 2, is guaranteed by a four-step handshake between the sender and receiver. This ensures that each message is delivered only once. In order to guarantee that each message is received only once, a four-step handshake is employed. This involves a connection request from one party being met with acceptance from the other before data transmission can take place. This process necessitates the generation of numerous acknowledgments, making it the most resource-demanding of all. In situations where the duplication of messages is not permitted, such as in financial transactions or critical control systems, QoS 2 is the dominant option. However, the reliability of QoS 2 comes at the expense of longer latency and greater bandwidth consumption due to the additional messaging overheads involved.

Mechanism

In QoS 2, the client and broker exchange a series of packets to ensure that the message is delivered exactly once:

1. The client sends a PUBLISH packet with a message ID to the broker.
2. The broker responds with a publish received (PubRec) packet, indicating it has received the message and is storing it.
3. The client sends a publish release (PubRel) packet to the broker, confirming that it can release the stored message.
4. The broker responds with a publish complete (PubComp) packet, indicating that the message has been processed, and the transaction is complete.

Use Cases

Typical use cases for QoS 2 include:

- Financial transactions where each message must be processed exactly once.
- Critical control systems where duplicate commands could cause erroneous behaviour.
- Applications requiring strict message integrity and cannot tolerate duplicates or loss.

Comparative Analysis

The choice of QoS level in MQTT is dictated by the specific requirements of the application. Each QoS level represents a trade-off between reliability and resource consumption:

- **QoS 0** offers the lowest latency and bandwidth usage but no delivery guarantee.
- **QoS 1** provides a balance between reliability and resource consumption, ensuring delivery with potential duplicates.
- **QoS 2** ensures exactly-once delivery at the cost of higher latency and bandwidth usage.

For the development of the system, the QoS 1 was selected, since it makes sure that no data is lost and is faster than QoS 2.

3.2 Python and Django for Web Development

Django is a Python-based web development framework that enables the rapid creation of web applications. The framework includes an SQLite database, authentication forms, a management panel, and a file upload feature, among others, to simplify web application development and avoid dependency issues commonly encountered in other frameworks [24].

Django is based on a Model-View-Template (MVT) architecture, which is a software design pattern for web development. The MVT structure is composed of three parts [24]:

1. **Model:** The Model serves as the interface for your data and is responsible for maintaining it. It represents the logical data structure of the entire application and is typically represented by a relational database such as MySQL or Postgres.
2. **View:** The View manages business logic, connects the Model and the Template, processes the HTTP requests, fetches and manipulates data and generate user responses .
3. **Template:** A template comprises fixed sections of the desired HTML/CSS/JavaScript (JS) and Jinja files output and special syntax that specifies how dynamic content will be inserted.

Key Features of Django

Some key features of Django include:

- **Object-Relational Mapping (ORM):** Django provides a powerful ORM that allows developers to interact with databases using Python code instead of SQL queries.
- **Automatic Admin Interface:** Django includes a fully customizable admin interface, which is generated dynamically based on your models.
- **Scalability:** Django is designed to handle high-traffic sites, making it scalable and suitable for a wide range of web applications.
- **Security:** Django helps developers avoid common security mistakes by providing a secure way to manage user authentication, passwords, and more.
- **Template System:** Django's templating system enables developers to create dynamic HTML pages with ease.

Chapter 4

Hardware and Software Development

This dissertation is divided in three main tasks

1. Subject-specific Knee Flexion Curve Generation
2. Instrumented Knee Brace development and implementation
3. Web application for diagnosis of human gait

In this chapter, this tasks will be explained with some detail, as well as the hardware and software used.

4.1 Subject-specific Knee Flexion Curve Generation

As each individual has a unique bone and muscle constitution, their knee flexion curve is also unique. By having that in mind, a deep learning model was developed to generate each person knee flexion curve using information about the person age, gender, weight (in kilogram (kg)) and height (in centimeters (cm)).

4.1.1 Data Selection

The main part of generation of the knee flexion curve using deep learning techniques is the data acquisition. By searching multiple public datasets, it was possible to collect 460 distinct curves that belong to only 31 individuals.

Three datasets were selected for use in this dissertation from the many that were analysed:

- Optitrack [25]

The first dataset comprises multiple gait sessions recorded for 10 subjects with varying characteristics. It contributes to the majority of the curves used for the generation curve data. Each subject performs 17 recordings, which are divided into 7 different exercises. Only the first 5 recordings of each subject represent normal gait. As each recording consists of multiple gait cycles, one recording has multiple knee flexion curves.

- Knee extension moment arm variations relate to mechanical function in walking and running [26]

The second dataset contain data on arm and knee flexion angles during walking and running, collected from 11 subjects. The purpose is to study the impact of knee flexion and arm flexion during walking and running exercises. Each subject performed one gait cycle for each exercise. Only the data representing knee flexion during one gait cycle was selected for our implemented knee curve generation model.

- Simultaneous Gait Data from Marker-based Stereophotogrammetry and 3D Temporal Scanner [27]

The third dataset records data from 12 subjects, similar to the previous two datasets. For each subject, information about joint angles for one gait cycle was collected. Only the knee flexion angle data was selected once it was the only data of interest.

The described datasets contained information about each patient and the corresponding knee flexion curve with 100 equally spaced points.

4.1.2 Model train and test

To perform the generation of the curve, a deep learning model was developed, based on a Multi Layer Perceptron (MLP) model.

The model consists of 3 linear layers with input value equal 4, that represents the number of input features and output value of 100, that represents the number of points in each curve from 0 to 100% of one gait cycle.

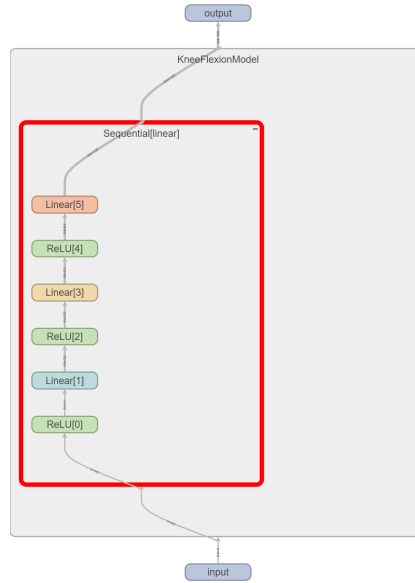


Figure 4.1: Model structure generated by using *tensorboard* python library

The data was divided into three distinct sets: a training set, a test set, and a validation set. The metrics used to assess the performance of the model during the training process were the error between the predicted output and the true output. The division was performed using a 75/15/10 ratio for the training set, validation set, and test set, respectively. After the division, the dataset was trained, and the resulting metrics for the training set, validation set (figure 4.2), and test set (figure 4.3) were as follows:

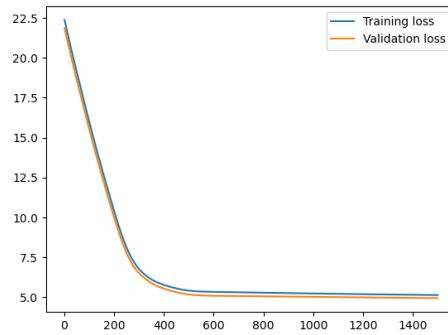


Figure 4.2: Training and Validation loss

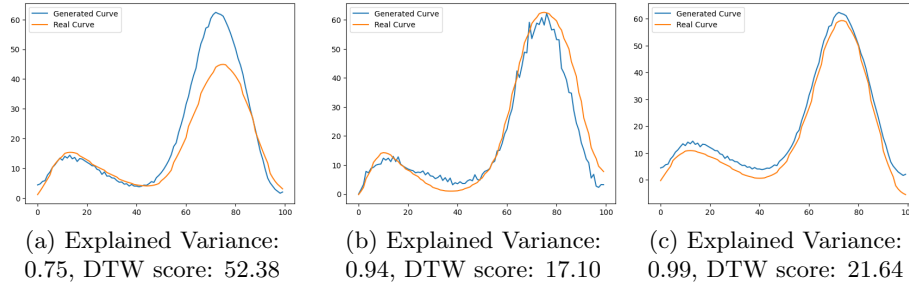


Figure 4.3: Comparison between generated and real curves from the test set

The metric used to validate the results during the test was Explained Variance [28] and DTW [29].

The Explained Variance Score is a statistical measure that indicates how well the model's predictions explain the variability of the actual data. It is calculated as follows:

$$\text{Explained Variance} = 1 - \frac{\text{Var}(y - \hat{y})}{\text{Var}(y)} \quad (4.1)$$

Where:

- y is the actual value.
- $\hat{y}$ is the predicted value.
- Var denotes the variance.

The score ranges from 0 to 1:

- A score of 1 indicates that the model perfectly explains the variance in the data.
- A score of 0 indicates that the model does not explain any of the variance.

The DTW algorithm is a method used to assess the degree of similarity between two time series sequences. The fundamental principle is to compute the distance between two points of correspondence between the time series sequences [29]. This is illustrated in figure 4.4, which compares DTW with the Euclidean distance.

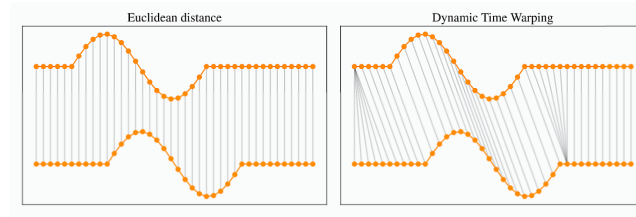


Figure 4.4: DTW vs Euclidean distance
[29]

A lower score for the DTW indicates a greater degree of similarity between the time series in question.

The results of the test set are presented in table 4.1.

Table 4.1: Test metrics scores

	Expected Variance	DTW
Mean	0.922	32.418
Standard Deviation	0.077	14.099
Minimum	0.757	17.102
Maximum	0.997	52.382

4.2 Instrumented Knee Brace

In order to develop the system, a choice had to be made about the hardware to be used in this dissertation. Various approaches were considered, such as the use of an IMU or whether to use a wireless approach or a cable connected to a computer. After some consideration, the solution chosen was to use wireless data transmission with an encoder to collect the data instead of an IMU sensor.

The development of the IKB was done together with my colleague Nuno Dias. The development process was divided between the two parties, where Nuno made the soldering of the necessary cables with the encoders and I managed the Arduino connection necessary for the solution chosen. There was a need to print some 3D supports, presented in figure 4.9, and for that the engineer João Lourenço helped with the process.

4.2.1 Arduino MKR Wi-Fi 1010

In order to implement a wireless data transmission approach, it was necessary to utilize a microcontroller board with a Wi-Fi connection module.

The Arduino MKR Wi-Fi 1010 is an appropriate selection for the methodology employed in this dissertation. The device is a low-power, compact board

based on the SAMD21 Cortex-M0+ 32-bit low-power ARM microcontroller. The device is a member of the Arduino MKR family, which has been developed for use in Internet of Things (IoT) applications.

Here are some characteristics about the Arduino MKR Wi-Fi 1010:

- **Microcontroller:** It is powered by the SAMD21G18A ARM Cortex-M0+ 32-bit low-power RISC processor, running at 48 MHz.
- **Memory:** It has 32 KB of SRAM and 256 KB of flash memory.
- **Connectivity:** The MKR Wi-Fi 1010 includes a U.FL connector for an external antenna and supports Wi-Fi 802.11 b/g/n connectivity.
- **Power Management:** It has an efficient power management system, allowing it to run on batteries or external power sources.
- **Input/Output:** The board has 22 digital input/output pins (including 8 PWM pins), 8 analogue input pins, and a UART, Serial Peripheral Interface (SPI), and I2C interface.
- **Programming:** It can be programmed using the Arduino IDE over a USB connection or wirelessly using the Arduino OTA (Over-The-Air) programming feature.
- **Form Factor:** The MKR Wi-Fi 1010 has a compact size (61.5×25 mm) and a castellated hole design, making it suitable for embedding into projects or prototypes.
- **Compatibility:** It is compatible with a wide range of Arduino shields and libraries.

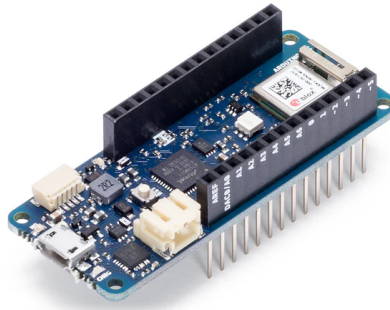


Figure 4.5: Arduino MKR Wi-Fi 1010

[30]

4.2.2 Encoder

An encoder is an electromechanical device that converts mechanical motion into electrical signals, thereby enabling the precise measurement and control of position, speed, and direction. There are several types of encoder, each with its own distinctive characteristics and applications. The following is a list of some of the more common types of encoder and their characteristics [31, 32, 33]:

- **Encoder technologies:**

- **Rotary Encoders**

- * Rotary encoders measure the rotational motion of a shaft or wheel.
 - * They can be incremental or absolute encoders.
 - * Incremental encoders output a series of pulses as the shaft rotates, allowing for relative position tracking.
 - * Absolute encoders provide a unique code for each angular position, enabling absolute position tracking without the need for a reference point.
 - * Rotary encoders are commonly used in robotics, industrial automation, and motion control systems.

- **Linear Encoders**

- * Linear encoders measure linear motion along a straight line or axis.
 - * They can be incremental or absolute, similar to rotary encoders.
 - * Linear encoders are often used in precision positioning systems, such as CNC machines, coordinate measuring machines (CMMs), and linear motion stages.

- **Encoder types:**

- **Optical Encoders**

- * Optical encoders use light sources (e.g., LEDs) and photodetectors to detect motion.
 - * They can be rotary or linear encoders.
 - * Optical encoders are known for their high resolution, accuracy, and reliability.

- * They are commonly used in applications that require precise motion control and positioning.

– **Magnetic Encoders**

- * Magnetic encoders use magnetic fields and sensors to detect motion.
- * They can be rotary or linear encoders.
- * Magnetic encoders are resistant to environmental factors like dust, moisture, and vibration, making them suitable for harsh environments.
- * They are often used in industrial automation, robotics, and automotive applications.

– **Capacitive Encoders**

- * Capacitive encoders use changes in capacitance to detect motion.
- * They are typically linear encoders and are known for their high resolution and accuracy.
- * Capacitive encoders are suitable for applications that require precise linear positioning, such as semiconductor manufacturing and precision metrology.

– **Inductive Encoders**

- * Inductive encoders use changes in inductance to detect motion.
- * They can be rotary or linear encoders.
- * Inductive encoders are robust and resistant to environmental factors like dust and moisture.
- * They are commonly used in industrial automation and harsh environments.

Following an in-depth analysis and comparison of the various encoder types, it was determined that a rotary encoder would be the optimal choice, given the necessity for angle measurement. With that in mind, the selected encoder was the CUI-AMT222B-V.

The CUI AMT222B-V capacitive absolute rotary encoder is a device that is widely utilised in a multitude of contexts where user input or control through rotation is necessary. These contexts include, but are not limited to, audio

equipment, industrial controls, robotics and human-machine interfaces (HMIs). The quadrature output enables the precise tracking of rotational direction and speed, rendering it an optimal choice for applications that necessitate accurate positional feedback or control. The following is a list of characteristics of the encoder:

- **Mechanical Characteristics**

- Shaft Type: Flatted
- Shaft Diameter: 6.35 mm
- Detent: 24 positions per revolution (15° detent angle)
- Rotational Life: 100,000 cycles (minimum)
- 12 to 14 bits of resolution

- **Electrical Characteristics**

- Output Type: Quadrature (two channels, A and B, with 90° phase shift)
- Output Circuit: Open collector
- Supply Voltage Range: 5 V $\pm$ 10%
- Output Current: 20 mA (maximum)

- **Mounting**

- Mounting Style: Panel mount
- Mounting Hole Diameter: 16 mm
- Mounting Depth: 25 mm

- **Environmental Characteristics**

- Operating Temperature Range: -20°C to +70°C
- Storage Temperature Range: -40°C to +85°C

- **Dimensions**

- Overall Diameter: 24 mm
- Overall Length: 25 mm



Figure 4.6: CUI AMT222B-V
[34]

The encoder use SPI protocol to communicate between multiple encoders. The set-up process for this protocol to enable communication between two sensors was a challenge, since the communication speed had to be compatible with the Arduino processor clock speed to make possible the readings of two encoders at the same time.

4.2.3 Knee Brace

To affix the encoder to the individual's knee, a knee brace was procured. The “Joelheira Ligamentar com Reforços Laterais Genu Dynastab” (Figure 4.7) was selected due to its cost-effectiveness, and its dual lateral reinforcements, which inhibit lateral knee movement.

The reinforcement structures, illustrated in Figure 4.8, are composed of three hinged, rigid plastic components. The two larger structures are connected to the central piece at two distinct points, thereby providing each reinforcement with two distinct rotation axes. When the lower axis is blocked, the system retains 115 degrees of freedom, which exceeds the necessary requirements for this application.

4.2.4 System Development

After obtaining all the necessary hardware, we brought together all the components to create a working IKB. This system is divided into two main components, the knee brace which contains the encoder and a box which will contain the Arduino board and the rest of the components. To attach the encoder to the knee brace, a support was designed. The support is made up of 4 components



Figure 4.7: Knee Brace with Lateral Reinforcements
[35]



Figure 4.8: Knee Brace Lateral Reinforcements

designed and produced using a 3D printer.

The purpose of these printed components is to support the encoder and rotate it in accordance with the movement of the knee while the patient is walking.

In order to achieve this, the shaft and the grommet are fitted together, and the joint is subjected to a slight degree of melting, thus ensuring that they adhere to each other. Subsequently, the newly constructed piece is affixed above the upper axis of rotation of the middle piece of the reinforcement. This design enhances the stability of the shaft. The third piece is affixed and fastened to the upper component of the reinforcement. The encoder is attached to this component with two screws measuring 2.5 mm in diameter, positioned to align with the shaft. This configuration enables the encoder to extract the knee angle. Finally, the fourth component is positioned between the middle and lower sections of the rear reinforcement. Its function is to restrict the movement of the lower component.

The subsequent step is to establish all the requisite connections to facilitate data acquisition via the encoder and the Arduino board. To connect the encoder to the Arduino board, we are using one RJ12 cable and two RJ12 connectors. It was necessary to attach some cables to the connectors creating, this way, a

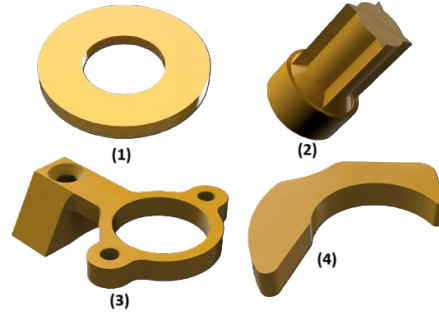


Figure 4.9: 3D Printed Components: (1) Grommet, (2) Shaft, (3) Encoder Support, (4) Movement Blocker

colour system for the connection between the encoder and the Arduino

Table 4.2: Pin Connections between Encoder, Arduino MKR 1010 and RJ12 Connector

Function	Encoder Pin	RJ12 Pin	Arduino Pin	Colour
+5 V	1	4	VCC	Black
SCLK	2	6	SCL	Pink
MOSI	3	3	MOSI	Orange
GND	4	5	GND	Yellow
MISO	5	1	MISO	Green
CHIP SELECT	6	2	D2	Blue

This colour system helps the maintenance and repair procedures of the Instrumented Knee Brace if necessary.

Once the colour system had been defined and both connectors connected to the encoder and the Arduino, the final system was assembled. The system is composed of the knee brace with the encoder attached, a box containing the Arduino board, a power supply, a green LED and a buzzer. The power supply employed in the system hardware development was a 5000mA power bank connected via a micro-USB cable to the Arduino board. The selected power system was chosen for two reasons: Firstly, the power banks were compact, measuring $8.5 \times 6 \times 1$ cm. Secondly, the possibility of easily replacing them when they ran out of power was a significant advantage.

Figure 4.10 depicts the system architecture. The blue rectangle represents the box used to house all the components, while the yellow box represents the knee brace attached with the encoder.

Figure 4.11 shows the fully developed system.

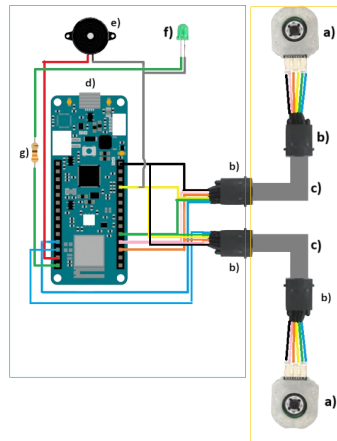
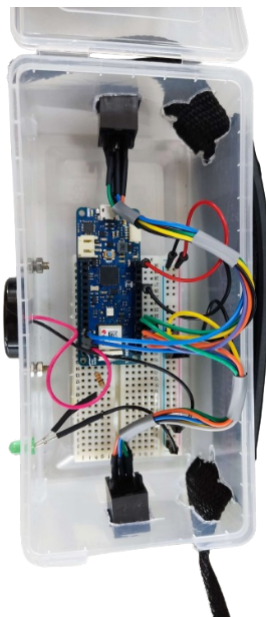


Figure 4.10: System Architecture: (a) encoder, (b) RJ12 Connector, (c) RJ12 Cable, (d) Arduino MKR 1010, (e) Buzzer, (f) Green LED, (g) 330Ω Resistor



(a) Arduino Box



(b) Right IKB



(c) Left IKB

Figure 4.11: Final system

4.3 Data Acquisition and Transmission System

The implementation of a data acquisition and transmission system is based on the use of an Arduino microcontroller. The system is designed to read angular position data from an encoder and transmit these data to an MQTT broker over a Wi-Fi network. The implementation leverages the Arduino Integrated Development Environment (IDE) and various libraries for MQTT communication and Wi-Fi connectivity. The primary component of the system are:

- Arduino MKR Wi-Fi 1010
- AMT22B Encoder
- MQTT Broker

The implementation of the system begins with the inclusion of the necessary libraries for MQTT and SPI communication, and Wi-Fi connection. Several macros were defined like the subscription and publisher topic as well as pin assignment for the encoder and some commands for the same.

```

1  #define BAUDRATE 9600
2
3  #define AMT22_NOP 0x00
4  #define AMT22_RESET 0x60
5  #define AMT22_ZERO 0x70
6
7  #define NEWLINE 0x0A
8
9  #define RES12 12
10 #define RES14 14
11
12 #define ENC_0 2 // Adjustment to the appropriate pin on the MKR 1010
13 #define ENC_1 3
14 #define \ac{SPI}_MOSI MOSI
15 #define \ac{SPI}_MISO MISO
16 #define \ac{SPI}_SCLK SCK
17
18 #define SUB_TOPIC "django/confirm"
19 #define PUB_TOPIC "django/gait_values_right"
20 #define PUB_TOPIC_1 "django/gait_values_left"
21
22 #define BUZZER_PIN 4
23 #define LED_PIN 5
24

```

Following the definition of all macros and libraries, the requisite functions for the configuration of Wi-Fi and MQTT were implemented. The Wi-Fi connection function attempts to establish a connection with a Wi-Fi network utilising the supplied SSID and password. Upon successful connection, another function handles the connection with the MQTT broker, employing the broker port number and address.

Once the Wi-Fi and MQTT configurations have been completed, it is necessary to implement functions to receive and process the encoder readings. The primary function reads the encoder position and verifies the data using a checksum. Once the data has been validated, it is converted from raw values to degrees. Additional functions handle SPI communication.

Upon completion of all requisite configurations, the system is prepared to transmit and receive data from the MQTT broker. Two functions were implemented to facilitate the transmission of data to and from the broker. The primary program comprises two principal functions: the setup function and the loop function.

The setup function initiates the system, establishes a connection to the Wi-Fi network, and subscribes to the MQTT topic. Additionally, it configures the SPI communication and sets the encoder's zero position.

The loop function is responsible for continuously examining the MQTT broker for incoming messages and for facilitating their transmission. Upon receipt of a message from the broker, the system initiates the data acquisition session. During this session, data is read from the encoder until a message is received, at which point the data collection from the MQTT broker is terminated, or the session is terminated after 20 seconds. Subsequently, the data is published to the MQTT broker. To prevent data loss, the Arduino saves the data collected from the encoders in batches of 20 values and transmits the batch as a whole, rather than each value individually. In the event of an untimely termination of the data acquisition session, the acquired values are transmitted to the MQTT broker.

The commencement and conclusion of the data acquisition process are indicated by an auditory signal, thus informing both the user and the patient that the session has commenced or concluded. In order to inform the user that all data has been successfully received by the web application, a musical tone is played once the Arduino has received a confirmation message from the web application.

For this dissertation, a QoS equal 1 was selected.

4.4 Web Application

In order to facilitate interaction between the hardware and the user and to present the results obtained from the hardware and the developed neural network, a web application was developed. This application was developed using the Django framework, as it already has a database and structure integrated, making it easier to create a web application.

To create the application, the following technologies were used

- Python Django
- HTML
- CSS
- JS

Since it was necessary to learn from scratch how to use the HTML, CSS and JS languages, this was the hardest part of the dissertation to complete.

The database used is SQLite, accessed via the sqlite3 library. SQLite is an embedded SQL database engine that is self-contained, highly reliable, full-featured and public domain. It is the most widely used database engine for web applications globally [36].

The web application is divided in three main pages

- Login
- Doctor page
- Patient information page

The database stores the information pertaining to each doctor, which is then retrieved by the application each time a login is attempted. Upon successful authentication, the user's main page, represented in figure 4.12, is displayed, from which access to each patient information page is permitted.

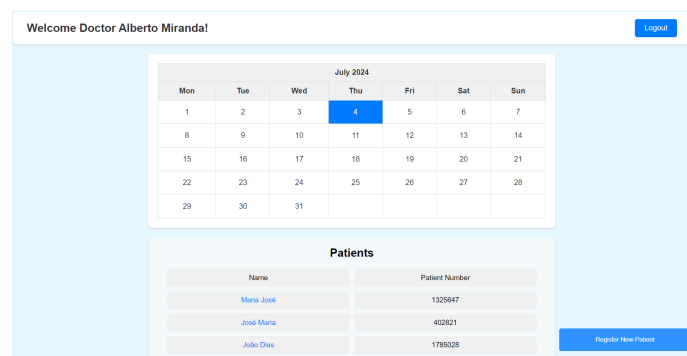


Figure 4.12: User main page

The patient information page, shown in figure 4.13, comprises a table of all sessions, ordered chronologically, a brief description of the patient's personal details, and a button to initiate a new session.

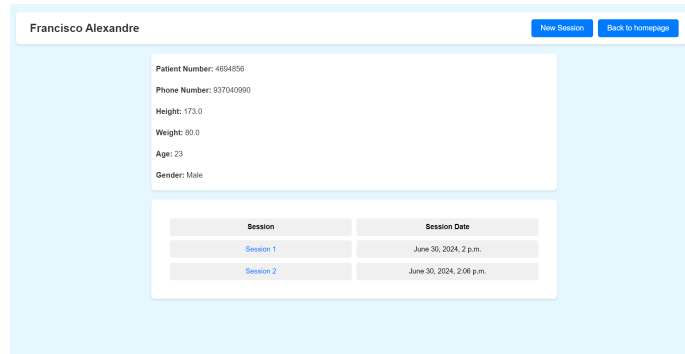


Figure 4.13: Patient information page

To review the preliminary sessions, the user must select one of the sessions displayed in the table on the patient information page, which will redirect them to a session review page, figure 4.14. This page includes the session gait graph and the 'normal' gait curve, which was obtained using the developed neural network.

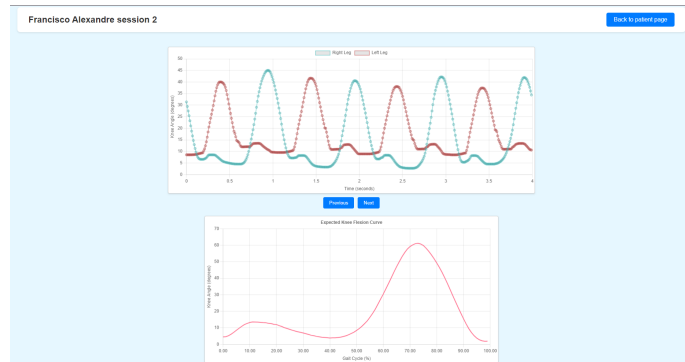


Figure 4.14: Session review page

To initiate a new session of gait analysis, the user must press the “New Session” button located on the patient information page. This action redirects the user to a new session page that contains a brief description of the session procedure along with four buttons: “Begin Session,” “End Session,” “Check Progress,” and “Back to Patient Page”, as presented in figure 4.15.

Upon selecting the “Begin Session” button, a message is transmitted to the Arduino via MQTT, and a pop-up message is displayed to confirm the commencement of a new session. Once the session has begun, the user must press the “End Session” button to conclude the gait analysis session. This action sends a message to the Arduino via MQTT to indicate the end of data collec-

tion, and a pop-up message confirms the termination of the session.

The “Check Progress” button allows the user to verify the progress of data collection between the broker and the web application.

To prevent simultaneous initiation of multiple sessions or the termination of a non-existent session, a control flag is incorporated into the web application software. This control mechanism helps to avoid potential errors and complications. Additionally, to ensure complete data collection, the user is not allowed to return to the patient information page once a session has started. This restriction prevents confusion and potential issues.

After all data is transmitted to the database, a message is sent to the IKB, that will emit a melody and permitting the user to return to the patient information page.

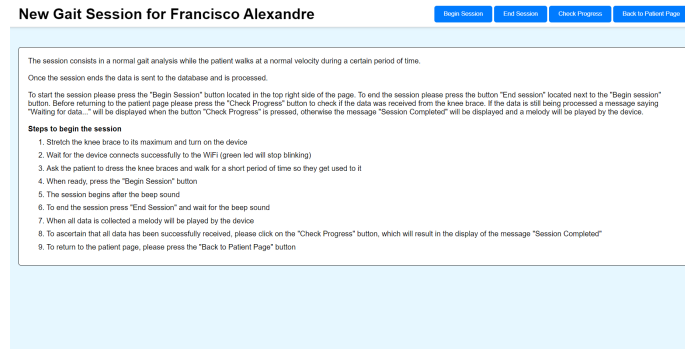


Figure 4.15: New session page

Chapter 5

Test and Results

Once the system had been fully developed, a series of tests were conducted to ascertain the precision and reliability of the data collected by the Instrumented Knee Brace.

The first step was to test the reliability of the encoders to be sure they were out of calibration.

Once the calibration of the encoders was in place, the whole system was tested.

5.1 Test procedure

The initial stage of the testing process was to define the mode of procedure.

Mode of Operation

1. The patient must dress the IKB and the user must turn on the device
2. The user must access the corresponding patient page and press “New Session”
3. After hearing a beep sound, the patient must walk during a maximum period of 20 seconds
4. Once the session is finished, the patient may remove the IKB and the user must wait until a melody to be played to be able to review the session

The procedure of operation will be divided into three phases: the preparation phase, the data collection phase, and the ending phase to better understand each

step.

Testing procedure

• Preparation phase

- It is necessary to extend the knee brace to its fullest extent in order to ascertain the zero-degree position.
- During the process of extending the knee brace, it is necessary to activate the Arduino board in order to set the zero position of the encoder.
- Once the green LED has ceased blinking, the patient is required to dress the IKB.
- In order to facilitate patient acclimation to the device, it is recommended that they engage in ambulation for approximately two minutes.
- Prior to commencing the session, the user must log in to the application and select the patient who will be participating.
- In order to commence a new session, the user is required to select the button labelled “New Session” on the patient information page.

• Data collection phase

- Once the user is prepared to initiate the data collection process, they must press the “Begin Session” button, playing a beep sound in the IKB.
- Upon hearing a beep sound, the patient is required to walk until a subsequent beep sound is heard 20 seconds prior to the beginning of the session.
- Upon the user’s intention to conclude the data collection process earlier, they must press the “End Session” button, which will result in the emission of a beep sound.

• Ending phase

- Following the conclusion of a session, the user is required to await the playback of a melody and subsequently press the “Check Progress” button to confirm the successful receipt of all data.
- Upon hearing the melody, the patient is able to remove the knee brace.

- Once data collection has been validated, the user is able to return to the patient information page and undertake an analysis of the session.

All pages layouts are presented in appendix A

5.2 Calibration

Prior to testing, it is necessary to identify the potential for software and hardware-related problems. Errors such as the calibration of the encoders and persistent readout errors, including the display of invalid values, must be considered. To ensure that both the IKB were calibrated, a small test code was developed to retrieve the values obtained by the encoders in four different positions and compare them with a protractor.

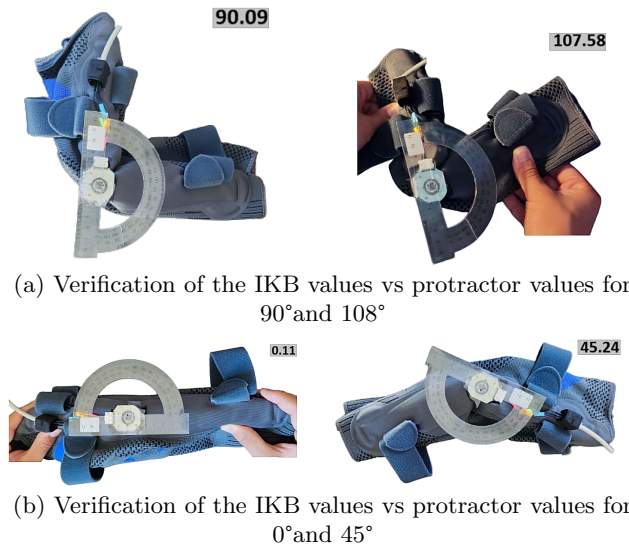


Figure 5.1: Calibration procedure

The values obtained from the encoder and from the protractor were the following

Table 5.1: Obtained angle values

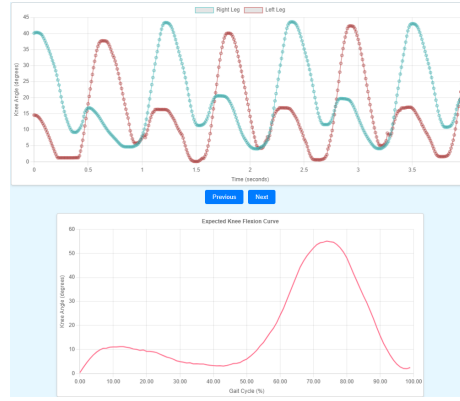
IKB Values	Protractor Values
0.11°	0°
45.24°	45°
90.09°	90°
107.58°	108°

The difference between the IKB readings and the protractor values is $\pm 0.25^\circ$. This error is between the range expected and does not affect the reliability of the data collected during the gait session.

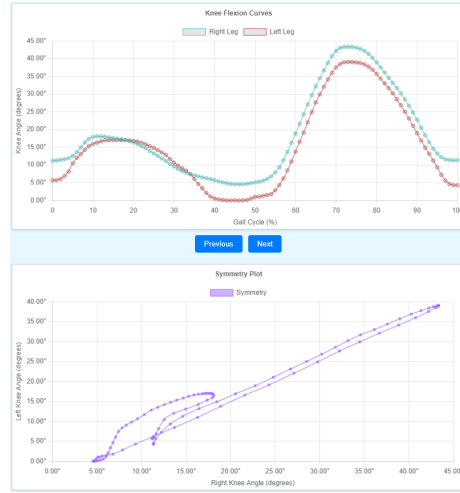
5.3 Tests

The first test conducted with the system fully operational involved a male subject, aged 57, with a body mass of 60.5 kg and a height of 166 cm. The subject was observed to ambulate for 20 seconds at a velocity that was within the normal range for the subject in a room where a change of direction was required. The results, which can be accessed via the subject information page on the web application, are as follows in figure 5.2.

The values obtained were within the expected range, although a minor discrepancy of approximately 2 to 5 degrees was evident. This discrepancy can be attributed to the anatomical characteristics of the subject, as no human has perfectly symmetric knees. This result allows the user to compare both knees and see if the values obtained fall within the range of the generated curve, which is shown below for reference.



(a) Session and generated curves

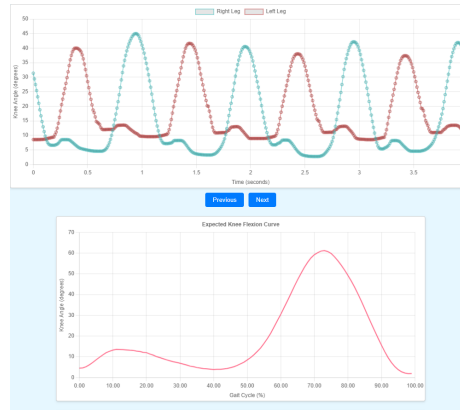


(b) Curve comparison

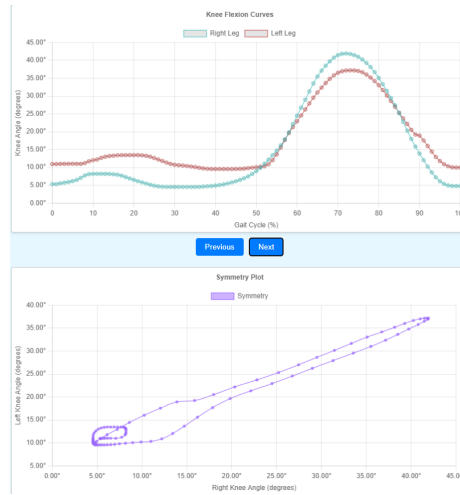
Figure 5.2: Session results and curve comparisons for first subject

This subject presents uniform gait cycles during all session with small deviation between steps. Both knees show consistent results, being the max flexion and lowest flexion values similar for all session.

The following test was conducted on a male subject, aged 23, with a body mass of 80 kg and a height of 173 cm. In accordance to the first test conducted, the session had a duration of 20 seconds at a velocity that was within the normal range for the subject in the same environment. This subject presented a misalignment on the formation of their knee and feet. The results are the ones shown in figure 5.3



(a) Session and generated curves



(b) Curve comparison

Figure 5.3: Session results and curve comparisons for second subject

The second subject exhibited some inconsistencies during the gait session, which can be attributed to the malformation of their knees and feet. In contrast to the first subject, the gait phases are not particularly explicit in each gait cycle. This suggests that the stance phase is characterised by rapid flexion and extension. This may also be attributed to the aforementioned malformations of the knees and feet.

The third test conducted included a female subject, aged 49, with a body mass of 89 kg and a height of 169 cm. In accordance to the first and second test conducted, the session had a duration of 20 seconds at a velocity that was within the normal range for the subject in the same environment. A notable degree of weakness was observed in the lower limbs during ambulation that were

noticed in the results obtained after the session presented in figure 5.4.

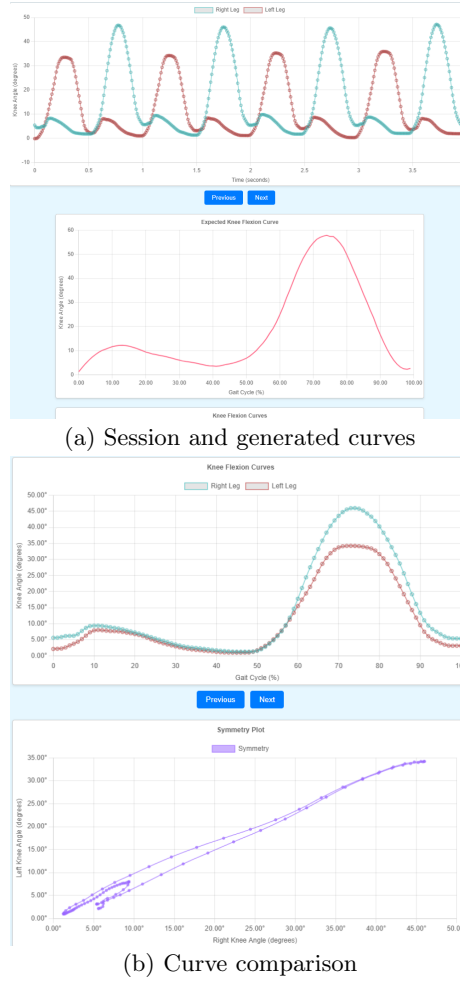


Figure 5.4: Session results and curve comparisons for third subject

The observed weakness in the lower limbs is reflected in the results obtained by the difference between the left and right knees. A lower flexion angle was observed during the swing phase on the left knee compared to the right knee. This could be attributed to the limping motion observed during the session.

Upon analysis of the results obtained from these tests, it can be concluded that the system is functioning within the expected parameters. It is evident that each subject's gait pattern differs, influenced by their unique body structure and age. Furthermore, each generated curve by the neural network model is distinct from each other.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This dissertation had the goal to develop a low-cost and low-power consumption wireless system that could measure the variation of the knee flexion angle during a gait session. For that, an encoder and an Arduino development board were used.

The calibration process of the encoders was not easy, since the SPI communication with two encoders brought some challenges. After some time to study the problem, a configuration that permitted the use of both encoders at the same time was achieved. After calibrating the SPI, the MQTT message exchange gave some drawbacks, since the initial idea for the dissertation was the real-time transmission of data to the developed web application. The MQTT protocol showed some limitations but, as well as the SPI, a solution was achieved.

One of the biggest challenges of this dissertation was the development of the web application, as there was a need to learn all web development languages (JS, HTML and CSS) and how to work with the Django framework.

Following the development and verification of the system's efficacy, a series of tests were conducted with three subjects, each exhibiting distinct characteristics and residing in disparate environments. Initially, some new calibrations were made, and communication problems were solved, but in the end the results were within the expected range.

Ultimately, a system was developed that is capable of collecting data on the variation of the knee angle during the gait process and processing the same data to present it in graphical form. This system enables the comparison of each cycle of both knees, synchronised in time, and their symmetry.

In general, the work developed showed promising and satisfactory results, where the main objective was to create a system with a wireless IKB and a diagnostic application that permitted the analysis of each patient gait sessions.

6.2 Future Work

The system developed showed promising results for the creation of a diagnosis system that can be used in a medical environment, but there are some improvements that can be applied.

The MQTT communication system presents certain limitations in the context of this dissertation. These constraints prevent the real-time transmission of values to the web application developed. Consequently, it is necessary to investigate alternative communication protocols, such as Web Sockets, with the objective of implementing a solution that enables real-time data transfer.

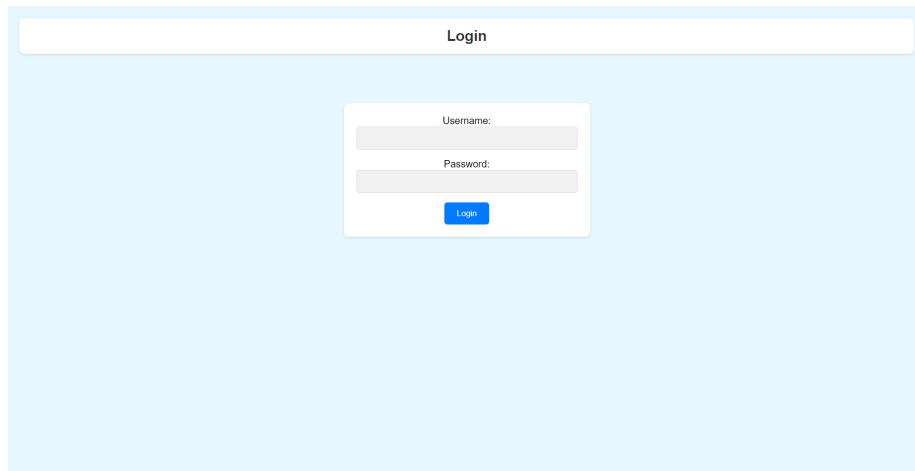
New input features could be added to the generation of knee flexion curves to make it more reliable and more unique, like stride velocity, stride length or even the distance between knees. It would also be interesting to use the rotation of the knee during the gait cycle, which implies another type of IKB.

Adapting the system for knee analysis during running exercises would also be interesting with the objective of providing more information to assist on diagnosis.

It would also be interesting the development of a IKB that permits a wireless data transmission between the Arduino board and the encoder.

Appendix A

Web Application layout



The image shows a web application login page. It features a light blue background. At the top, there is a white header bar with the word "Login" centered. Below the header, in the center of the page, is a white login form. The form contains two input fields: "Username:" and "Password:". Below these fields is a blue button with the text "Login" in white.

Figure A.1: Login Page

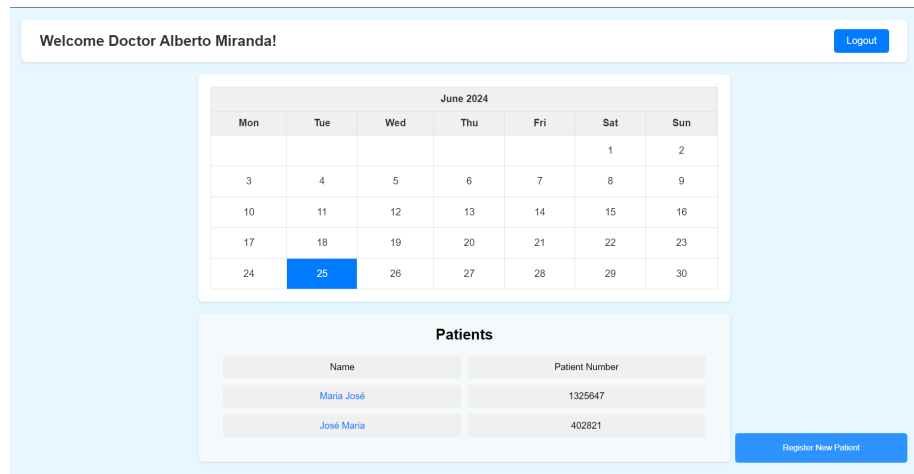


Figure A.2: Main User Page

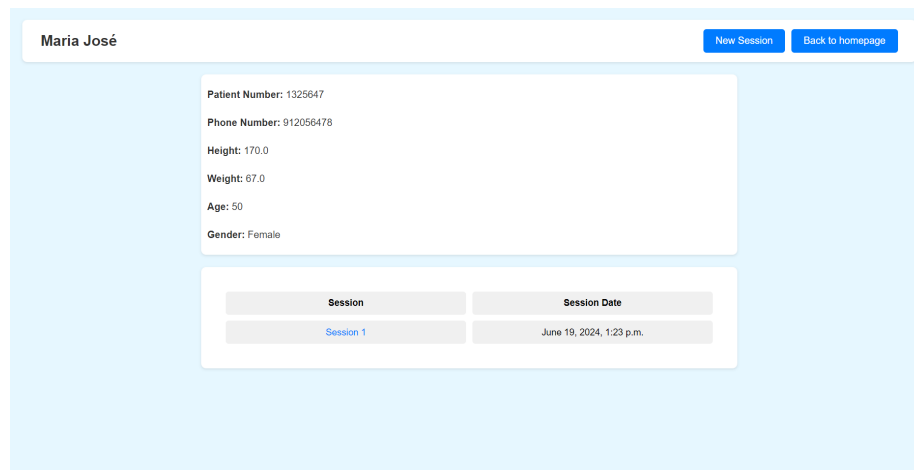


Figure A.3: Patient Information Page

New Gait Session for Maria José

[Begin Session](#)[End Session](#)[Check Progress](#)[Back to Patient Page](#)

The session consists in a normal gait analysis while the patient walks at a normal velocity during a certain period of time.

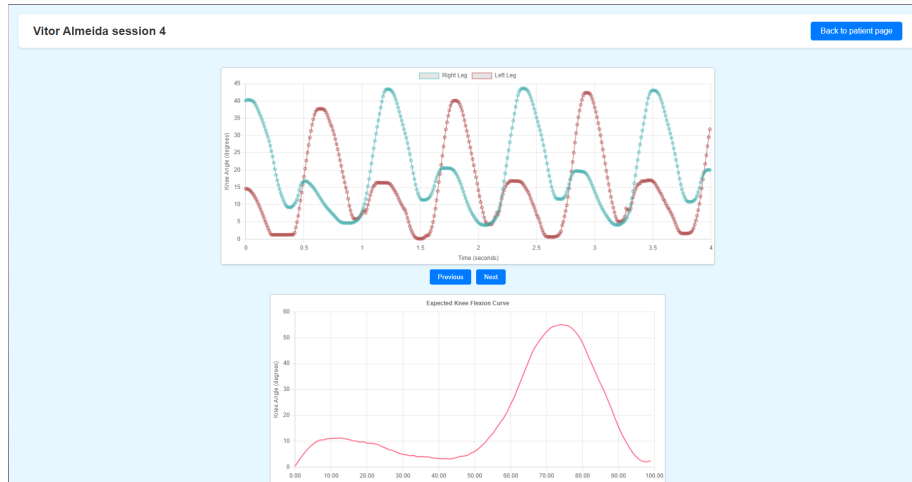
Once the session ends the data is sent to the database and is processed.

To start the session please press the "Begin Session" button located in the top right side of the page. To end the session please press the button "End session" located next to the "Begin session" button. Before returning to the patient page please press the "Check Progress" button to check if the data was received from the knee brace. If the data is still being processed a message saying "Waiting for data..." will be displayed when the button "Check Progress" is pressed, otherwise the message "Session Completed" will be displayed and a melody will be played by the device.

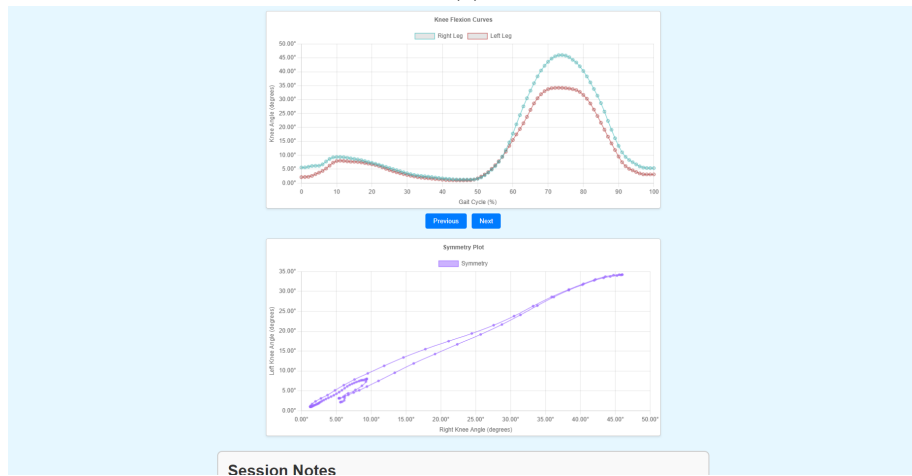
Steps to begin the session

1. Stretch the knee brace to its maximum and turn on the device
2. Wait for the device connects successfully to the WiFi (green led will stop blinking)
3. Ask the patient to dress the knee braces and walk for a short period of time so they get used to it
4. When ready, press the "Begin Session" button
5. The session begins after the beep sound
6. To end the session press "End Session" and wait for the beep sound
7. When all data is collected a melody will be played by the device
8. To ascertain that all data has been successfully received, please click on the "Check Progress" button, which will result in the display of the message "Session Completed"
9. To return to the patient page, please press the "Back to Patient Page" button

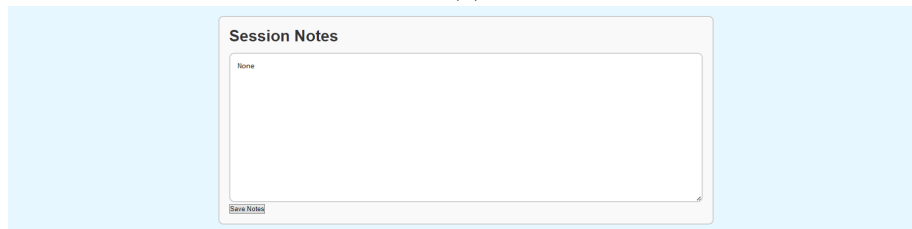
Figure A.4: New Session Page



(a)



(b)



(c)

Figure A.5: Session Review Page

Appendix B

Arduino code

```
1  #include <ArduinoMqttClient.h>
2  #include <SPI.h>
3  #include <WiFiNINA.h>
4  #include <stdbool.h>
5  #include <stdio.h>
6  #include <Arduino.h>
7  #include <string.h>
8  #include <CircularBuffer.hpp>
9
10 // Constants
11 #define BAUDRATE 9600
12
13 #define AMT22_NOP 0x00
14 #define AMT22_RESET 0x60
15 #define AMT22_ZERO 0x70
16
17 #define NEWLINE 0x0A
18
19 #define RES12 12
20 #define RES14 14
21
22 #define ENC_0 2
23 #define ENC_1 3
24 #define SPI_MOSI MOSI
25 #define SPI_MISO MISO
26 #define SPI_SCLK SCK
27
28 #define SUB_TOPIC "django/confirm"
29 #define PUB_TOPIC "django/gait_values_right"
30 #define PUB_TOPIC_1 "django/gait_values_left"
31
32 #define BUZZER_PIN 4
33 #define LED_PIN 5
34
35 #define left ENC_0
36 #define right ENC_1
```

```

37 // Define note frequencies (in Hz)
38 #define NOTE_C4 262
39 #define NOTE_D4 294
40 #define NOTE_E4 330
41 #define NOTE_F4 349
42 #define NOTE_G4 392
43 #define NOTE_A4 440
44 #define NOTE_B4 494
45 #define NOTE_C5 523
46
47 const long INTERVAL = 59990;
48 const int MAX_READINGS = 2000;
49 const int BATCH_SIZE = 20;
50 const int BUZZER_ON_DURATION = 1000;
51 const int NOTE_DURATION = 500;
52 const int PAUSE_BETWEEN_NOTES = 530;
53
54 unsigned int previousMillis = 0;
55 int count = 0;
56
57 struct data {
58     String degree;
59 };
60
61 bool permission = false;
62 bool session_end_flag = false;
63 int all_received = 0;
64
65 CircularBuffer<float, MAX_READINGS> reading_r;
66 CircularBuffer<float, MAX_READINGS> reading_l;
67
68 WiFiClient wifiClient;
69
70 class WiFiManager {
71 public:
72     bool connect(const char* ssid, const char* pass, int maxAttempts = 10) {
73         int attempts = 0;
74         while (WiFi.begin(ssid, pass) != WL_CONNECTED && attempts < maxAttempts) {
75             Serial.println("Attempting to connect to WiFi");
76             digitalWrite(LED_PIN, HIGH);
77             delay(1000);
78             digitalWrite(LED_PIN, LOW);
79             delay(100);
80             attempts++;
81         }
82         return WiFi.status() == WL_CONNECTED;
83     }
84 };
85
86 class MQTTManager {
87     MqttClient mqttClient;
88 public:
89     MQTTManager(WiFiClient& wifiClient) : mqttClient(wifiClient) {}
90
91     bool connect(const char* broker, int port, int maxAttempts = 10) {
92         int attempts = 0;
93     }

```

```

94     while (!mqttClient.connect(broker, port) && attempts < maxAttempts) {
95         Serial.print("MQTT connection failed! Error code = ");
96         Serial.println(mqttClient.connectError());
97         delay(1000);
98         attempts++;
99     }
100     return mqttClient.connected();
101 }
102
103 void publish(const char* topic, const String& data, int qos) {
104     mqttClient.beginMessage(topic, false, qos);
105     mqttClient.print(data);
106     mqttClient.endMessage();
107 }
108
109 void onMessage(void (*callback)(int)) {
110     mqttClient.onMessage(callback);
111 }
112
113 void poll() {
114     mqttClient.poll();
115 }
116
117 bool available() {
118     return mqttClient.available();
119 }
120
121 int read() {
122     return mqttClient.read();
123 }
124
125 String messageTopic() {
126     return mqttClient.messageTopic();
127 }
128
129 void subscribe(const char* topic) {
130     mqttClient.subscribe(topic);
131 }
132 };
133
134 WiFiManager wifiManager;
135 MQTTManager mqttManager(wifiClient);
136
137 bool Connect_to_WiFi(char ssid[], char pass[], int maxAttempts = 10) {
138     return wifiManager.connect(ssid, pass, maxAttempts);
139 }
140
141 void Connect_to_Broker(char broker[], int port, int maxAttempts = 10) {
142     if (mqttManager.connect(broker, port, maxAttempts)) {
143         Serial.println("You're connected to the MQTT broker!");
144         Serial.println();
145     } else {
146         Serial.println("Failed to connect to MQTT broker after multiple
147             ↪ attempts.");
148     }
149 }

```

```

150 void publisher(char topic[], String data, int qos) {
151     // if (session_end_flag == true) {
152     //     Serial.print("Sending data ");
153     //     Serial.print(data);
154     //     Serial.print("\n");
155     // }
156     mqttManager.publish(topic, data, qos);
157 }
158
159 void onMqttMessage(int messageSize) {
160     Serial.println("Received a message with topic ");
161     Serial.print(mqttManager.messageTopic());
162     Serial.print(", length ");
163     Serial.print(messageSize);
164     Serial.println(" bytes:");
165
166     char message[messageSize + 1];
167     int index = 0;
168
169     while (mqttManager.available() && index < messageSize) {
170         message[index++] = (char)mqttManager.read();
171     }
172     message[index] = '\0';
173
174     Serial.print("Message: ");
175     Serial.println(message);
176
177     if (strcmp(message, "Begin") == 0) {
178         permission = true;
179     } else if (strcmp(message, "End") == 0) {
180         permission = false;
181         session_end_flag = true;
182     } else if (strcmp(message, "Received") == 0) {
183         all_received = 1;
184         Serial.println(all_received);
185     }
186     Serial.println();
187     Serial.println();
188 }
189
190 uint16_t getPositionSPI(uint8_t encoder, uint8_t resolution) {
191     uint16_t currentPosition;
192     bool binaryArray[16];
193
194     currentPosition = spiWriteRead(AMT22_NOP, encoder, false) << 8;
195     delayMicroseconds(3);
196     currentPosition |= spiWriteRead(AMT22_NOP, encoder, true);
197
198     for (int i = 0; i < 16; i++) binaryArray[i] = (0x01 & (currentPosition >>
199         ↪ i));
200
201     if ((binaryArray[15] == !(binaryArray[13] ^ binaryArray[11] ^ binaryArray[9]
202         ↪ ^ binaryArray[7] ^ binaryArray[5] ^ binaryArray[3] ^ binaryArray[1]))
203         && (binaryArray[14] == !(binaryArray[12] ^ binaryArray[10] ^
204         ↪ ^ binaryArray[8] ^ binaryArray[6] ^ binaryArray[4] ^ binaryArray[2] ^
205         ↪ ^ binaryArray[0]))) {
206         currentPosition &= 0x3FFF;

```



```

203     } else {
204         currentPosition = 0xFFFF;
205     }
206
207     if ((resolution == RES12) && (currentPosition != 0xFFFF)) currentPosition =
        ↳ currentPosition >> 2;
208
209     return currentPosition;
210 }
211
212 uint8_t spiWriteRead(uint8_t sendByte, uint8_t encoder, uint8_t releaseLine) {
213     uint8_t data;
214
215     setCSLine(encoder, LOW);
216     delayMicroseconds(3);
217     data = SPI.transfer(sendByte);
218     delayMicroseconds(3);
219     setCSLine(encoder, releaseLine);
220
221     return data;
222 }
223
224 void setCSLine(uint8_t encoder, uint8_t csLine) {
225     digitalWrite(encoder, csLine);
226 }
227
228 void setZeroSPI(uint8_t encoder) {
229     spiWriteRead(AMT22_NOP, encoder, false);
230     delayMicroseconds(3);
231     spiWriteRead(AMT22_ZERO, encoder, true);
232     delay(250);
233 }
234
235 void resetAMT22(uint8_t encoder) {
236     spiWriteRead(AMT22_NOP, encoder, false);
237     delayMicroseconds(3);
238     spiWriteRead(AMT22_RESET, encoder, true);
239     delay(250);
240 }
241
242 float read_encoder(uint8_t encoder) {
243     uint8_t attempts = 0;
244     float encoderPositionDegree;
245     uint16_t encoderPosition = getPositionSPI(encoder, RES14);
246
247     while (encoderPosition == 0xFFFF && ++attempts < 3) {
248         encoderPosition = getPositionSPI(encoder, RES14);
249     }
250
251     if (encoderPosition == 0xFFFF) {
252         Serial.print("Encoder error. Attempts: ");
253         Serial.print(attempts, DEC);
254         Serial.write(NEWLINE);
255     } else {
256         encoderPositionDegree = encoderPosition / 45.5083;
257         if (encoder == right) {
258             encoderPositionDegree = 360 - encoderPositionDegree;

```

```
259     }
260     encoderPositionDegree = round(encoderPositionDegree * 100) / 100;
261     if (encoderPositionDegree > 180) {
262         encoderPositionDegree = encoderPositionDegree - 360;
263     }
264 }
265 return encoderPositionDegree;
266 }
267
268 void setup() {
269     char ssid[] = "Tunes";
270     char pass[] = "tunesblack";
271     char broker[] = "test.mosquitto.org";
272     int port = 1883;
273
274     pinMode(SPI_SCLK, OUTPUT);
275     pinMode(SPI_MOSI, OUTPUT);
276     pinMode(SPI_MISO, INPUT);
277     pinMode(ENC_0, OUTPUT);
278     pinMode(ENC_1, OUTPUT);
279     pinMode(BUZZER_PIN, OUTPUT);
280     pinMode(LED_PIN, OUTPUT);
281
282     Serial.begin(BAUDRATE);
283     digitalWrite(ENC_0, HIGH);
284     digitalWrite(ENC_1, HIGH);
285     SPI.begin();
286
287     setZeroSPI(ENC_0);
288     setZeroSPI(ENC_1);
289     SPI.beginTransaction(SPISettings(500000, MSBFIRST, SPI_MODE0));
290
291     while (!Connect_to_WiFi(ssid, pass)) {
292         Serial.println("Attempting to connect to WiFi");
293         digitalWrite(LED_PIN, HIGH);
294         delay(1000);
295         digitalWrite(LED_PIN, LOW);
296         delay(100);
297     }
298     Serial.print("Connected to WiFi");
299     digitalWrite(LED_PIN, LOW);
300     Connect_to_Broker(broker, port);
301
302     mqttManager.onMessage(onMqttMessage);
303
304     Serial.print("Subscribing to topic: ");
305     Serial.println(SUB_TOPIC);
306     Serial.println();
307
308     mqttManager.subscribe(SUB_TOPIC);
309
310     Serial.print("Waiting for messages on topic: ");
311     Serial.println(SUB_TOPIC);
312     Serial.println();
313 }
314
315 void loop() {
```

```

316     if (permission == true) {
317         Serial.print("Beginning session\n");
318         begin_reading();
319         delay(1000);
320         save_data();
321     } else {
322         if (session_end_flag == true) {
323             String count_buffer;
324             count_buffer = String("End");
325             publisher(PUB_TOPIC, count_buffer, 1);
326             publisher(PUB_TOPIC_1, count_buffer, 1);
327             session_end_flag = false;
328             delay(5000);
329         } else {
330             Serial.print("No session or data to be sent\n");
331             if (all_received == 1) {
332                 alld_data_received();
333             }
334             delay(1000);
335         }
336         mqttManager.poll();
337     }
338 }
339
340 void save_data() {
341     reading_r.clear();
342     reading_l.clear();
343     int readingsCount = 0;
344
345     while (permission == true && readingsCount < MAX_READINGS){
346         reading_r.push(read_encoder(ENC_0));
347         reading_l.push(read_encoder(ENC_1));
348         readingsCount++;
349         mqttManager.poll();
350         delay(10);
351     }
352
353     int index = 0;
354     begin_reading();
355     while (index < readingsCount) {
356         String buffer_r, buffer_l;
357         for (int i = 0; i < BATCH_SIZE && index < readingsCount; i++, index++) {
358             buffer_r += String(reading_r[index], 2);
359             buffer_l += String(reading_l[index], 2);
360
361             if (i < BATCH_SIZE - 1 && index < readingsCount - 1) {
362                 buffer_r += ",";
363                 buffer_l += ",";
364             }
365         }
366         publisher(PUB_TOPIC, buffer_r, 2);
367         publisher(PUB_TOPIC_1, buffer_l, 2);
368         if (permission == true) {
369             permission = false;
370             session_end_flag = true;
371         }
372     }

```

```
373 }
374
375 void begin_reading() {
376     digitalWrite(BUZZER_PIN, HIGH);
377     delay(BUZZER_ON_DURATION);
378     digitalWrite(BUZZER_PIN, LOW);
379 }
380
381 void alld_data_received() {
382     int melody[] = {
383         NOTE_C4, NOTE_D4, NOTE_E4, NOTE_F4,
384         NOTE_G4, NOTE_A4, NOTE_B4, NOTE_C5
385     };
386
387     int noteDurations[] = {
388         NOTE_DURATION, NOTE_DURATION, NOTE_DURATION, NOTE_DURATION,
389         NOTE_DURATION, NOTE_DURATION, NOTE_DURATION, NOTE_DURATION
390     };
391     Serial.print("\n\n\t\tIM HERE\n\n");
392     for (int thisNote = 0; thisNote < 8; thisNote++) {
393         int noteDuration = noteDurations[thisNote];
394         tone(BUZZER_PIN, melody[thisNote], noteDuration);
395         int pauseBetweenNotes = noteDuration + 30;
396         delay(pauseBetweenNotes);
397         noTone(BUZZER_PIN);
398     }
399     all_received = 0;
400 }
```

Bibliography

- [1] “The gait cycle: Phases, parameters to evaluate & technology — gait cycle analysis — tekscan.” <https://www.tekscan.com/blog/medical/gait-cycle-phases-parameters-evaluate-technology>. (Accessed on 07/05/2024).
- [2] A. Alharbi, K. Equbal, S. Ahmad, H. U. Rahman, H. Alyami, *et al.*, “Human gait analysis and prediction using the levenberg-marquardt method,” *Journal of Healthcare Engineering*, vol. 2021, 2021.
- [3] M. Abid, N. Mezghani, A. Mitiche, *et al.*, “Knee joint biomechanical gait data classification for knee pathology assessment: a literature review,” *Applied bionics and biomechanics*, vol. 2019, 2019.
- [4] J. Prathap Kumar, M. Arun Kumar, and D. Venkatesh, “Healthy gait: Review of anatomy and physiology of knee joint,” *International Journal of Current Research and Review*, vol. 12, no. 6, pp. 1–8, 2020.
- [5] “Common knee injuries - orthoinfo - aaos.” <https://orthoinfo.aaos.org/en/diseases--conditions/common-knee-injuries/>. (Accessed on 06/27/2024).
- [6] S. B. Kwon, Y. Ku, H.-S. Han, M. C. Lee, H. C. Kim, and D. H. Ro, “A machine learning-based diagnostic model associated with knee osteoarthritis severity,” *Scientific Reports*, vol. 10, no. 1, p. 15743, 2020.
- [7] “Gait analysis — orthopedic medical center.” <https://www.orthomedctr.com/gait-analysis.php>. (Accessed on 07/05/2024).
- [8] J. Farah, N. Baddour, and E. Lemaire, “Gait phase detection from thigh kinematics using machine learning techniques,” pp. 263–268, 05 2017.
- [9] R. B. Davis, “Reflections on clinical gait analysis,” *Journal of Electromyography and Kinesiology*, vol. 7, no. 4, pp. 251–257, 1997.

- [10] N. Helwig and E. Hsiao-Wecksler, “Multivariate Gait Data.” UCI Machine Learning Repository, 2022. DOI: <https://doi.org/10.24432/C5861T>.
- [11] Vicon, *Vicon Motion Capture Systems*. Oxford: Vicon Motion Systems Ltd., 1984.
- [12] D. A. Winter, *The Biomechanics and Motor Control of Human Gait: Normal, Elderly and Pathological*. Waterloo: University of Waterloo Press, 1991.
- [13] D. Jáuregui, *3D motion capture by computer vision and virtual rendering*. PhD thesis, 05 2011.
- [14] S. L. Delp, J. P. Loan, M. G. Hoy, F. E. Zajac, E. L. Topp, and J. M. Rosen, “An interactive graphics-based model of the lower extremity to study orthopaedic surgical procedures,” *IEEE Transactions on Biomedical Engineering*, vol. 37, no. 8, pp. 757–767, 1990.
- [15] J. Perry, *Gait Analysis: Normal and Pathological Function*. Thorofare: Slack Incorporated, 1992.
- [16] H. Wu and J. Wang, “A vision-based gait recognition method for human identification using deep learning,” *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 1, pp. 45–55, 2018.
- [17] F. Li, X. Liu, and J. Chu, “Automated gait analysis based on deep learning,” *Journal of Biomedical Informatics*, vol. 105, pp. 103–113, 2020.
- [18] Y. Nasser, R. Jennane, A. Chetouani, E. Lespessailles, and M. E. Hassouni, “Discriminative regularized auto-encoder for early detection of knee osteoarthritis: Data from the osteoarthritis initiative,” *IEEE Transactions on Medical Imaging*, vol. 39, no. 9, pp. 2976–2984, 2020.
- [19] T. Seel, J. Raisch, and T. Schauer, “Imu-based joint angle measurement for gait analysis,” *Sensors*, vol. 14, no. 4, pp. 6891–6909, 2014.
- [20] L. S. Vargas-Valencia, A. Elias, E. Rocon, T. Bastos-Filho, and A. Frizera, “An imu-to-body alignment method applied to human gait analysis,” *Sensors*, vol. 16, no. 12, 2016.
- [21] G. Domínguez, E. Cardiel, S. Arias, and P. Rogeli, “A digital goniometer based on encoders for measuring knee-joint position in an orthosis,” in *2013 World Congress on Nature and Biologically Inspired Computing*, pp. 1–4, 2013.
- [22] “What is the mqtt protocol: A beginner’s guide — emq.” <https://www.emqx.com/en/blog/>

- `the-easiest-guide-to-getting-started-with-mqtt`. (Accessed on 04/05/2024).
- [23] “Mqtt qos 0, 1, 2 explained: A quickstart guide — emq.” <https://www.emqx.com/en/blog/introduction-to-mqtt-qos>. (Accessed on 04/05/2024).
- [24] GeeksforGeeks, “Django basics,” Apr 2023. (Accessed on 07/05/2024).
- [25] S. Gonzaacute;lez, “Gait dataset using a 3d motion capture system,” Sep 2021. (Accessed on 12/05/2023).
- [26] M. Wheatley, “Knee extension moment arm variations relate to mechanical function in walking and running,” Aug. 2021.
- [27] A. V. Ruescas Nicolau, H. De Rosario, F. Basso Della-Vedova, E. Parrilla Bernabé, M.-C. Juan, and J. López-Pascual, “Accuracy of a 3d temporal scanning system for gait analysis: Comparative with a marker-based photogrammetry system,” *Gait Posture*, vol. 97, pp. 28–34, 2022.
- [28] “Explained variance — pytorch-metrics 1.4.0.post0 documentation.” https://lightning.ai/docs/torchmetrics/stable/regression/explained_variance.html#module-interface. (Accessed on 07/04/2024).
- [29] “Dynamic time warping(dtw) algorithm in time series.” <https://www.theaidream.com/post/dynamic-time-warping-dtw-algorithm-in-time-series>. (Accessed on 07/04/2024).
- [30] “Arduino mkr wifi 1010 — arduino online shop.” <https://store-usa.arduino.cc/products/arduino-mkr-wifi-1010>. (Accessed on 06/24/2024).
- [31] “11 different types of encoders for dc motors — assum motor.” <https://assummotor.com/blog/types-of-encoders/>. (Accessed on 07/29/2024).
- [32] “Encoders: Types of encoders & classes — arrow.com.” <https://www.arrow.com/en/research-and-events/articles/how-to-understand-the-different-types-of-encoders>. (Accessed on 07/29/2024).
- [33] “Difference between rotary and linear encoders - vitruve.” <https://vitruve.fit/blog/difference-between-rotary-and-linear-encoders/>. (Accessed on 07/29/2024).

-
- [34] “Amt222b-v — modular — absolute — rotary encoders.” <https://www.cuidevices.com/product/motion-and-control/rotary-encoders/absolute/modular/amt222b-v>. (Accessed on 06/24/2024).
- [35] M. Shop, “Joelheira ligamentar com reforços laterais genu dynastab.” https://www.medicalshop.pt/joelheira-ligamentar-com-reforc-os-laterais-ligaflex-or0802047/p?gclid=Cj0KCQiAgK2qBhCHARIsAGACuzmf4hNr_y1XmVkd5m6Lmtvmi-CiXUTHDVEmMuqMoSFd7Yzyr9VbKDY8aArkGEALw_wcBidsku=14517&skuId=14518. (Accessed on 06/24/2024).
- [36] “Introduction to sqlite in python - geeksforgeeks.” <https://www.geeksforgeeks.org/introduction-to-sqlite-in-python/?ref=lbp>. (Accessed on 06/24/2024).